

DNS y DNSSEC para Operadores

Carlos Martinez-Cagnazzo

Nicolás Antoniello

2026-08-07

Carlos Martinez-Cagnazzo
LACNIC, Montevideo, Uruguay – <https://www.lacnic.net>
ORCID: <https://orcid.org/0000-0001-8828-5239>
carlos@lacnic.net

Nicolás Antoniello
nantoniello@gmail.com

Este curso ofrece una introducción práctica a DNSSEC orientada a operadores de redes pequeños y medianos cubriendo desde los fundamentos de firma y validación hasta la gestión de claves con KASP en BIND. A lo largo de varios módulos aborda el monitoreo y la resolución de problemas, arquitecturas de firmante oculto (hidden signer) y temas avanzados como NSEC3, rollover de algoritmos, CDS/CDNSKEY y configuraciones multi-firmantes.

Table of contents

1	Introducción	7
2	Temario	8
2.1	1. Introducción a DNSSEC	8
2.2	2. Los registros de DNSSEC	8
2.3	3. Zonas autoritativas con BIND	8
2.4	4. Firmando una zona con BIND y KASP	9
2.5	5. Monitoreo y troubleshooting	9
2.6	6. Arquitectura con hidden signer	9
2.7	7. DNSSEC avanzado	9
3	01 - Introducción a DNSSEC	11
3.1	Repaso breve de DNS	11
3.2	Fundamentos de criptografía	11
3.2.1	Criptografía	12
3.2.2	Criptografía de clave pública	12
3.2.3	Hashes (digest / resúmenes)	12
3.2.4	Firma digital	14
3.3	¿Qué es DNSSEC?	15
3.4	¿Qué problemas resuelve?	16
3.5	¿Qué NO resuelve DNSSEC?	16
3.6	Cadena de confianza	17
3.7	Conceptos clave	19
3.8	Resumen	19
4	02 - Los registros de DNSSEC	20
4.1	DNSKEY — la clave pública de la zona	20
4.2	RRSIG — la firma sobre un RRset	21
4.3	DS — el enlace hacia el padre	22
4.4	NSEC — negar que algo no existe, de forma autenticada	22
4.5	Resumen	23
5	03 - Zonas autoritativas con BIND	24
5.1	Instalando BIND 9 en Ubuntu	24
5.2	Repaso del layout de configuración en Ubuntu	25
5.3	Configurando rndc	26

5.4	Crear una zona autoritativa primaria	28
5.5	Verificar el funcionamiento con dig/drill	29
5.6	Configurar una zona autoritativa secundaria	30
5.6.1	Permitir AXFR/IXFR de forma segura y controlada	33
5.7	Consideraciones de seguridad	34
5.7.1	Prevenir la recursión no deseada	34
5.7.2	Prevenir las respuestas tomadas de caché no deseadas	34
5.7.3	Prevenir las transferencias de zona no deseadas	35
5.8	Comentarios sobre configurar los NS en la zona padre	35
5.9	Resumen	36
6	04 - Firmando una zona con BIND y KASP	37
6.1	Repaso: KSK vs ZSK, y por qué el rollover	37
6.2	Qué es KASP	37
6.3	Migrar el primario a firmado inline	38
6.4	Configurando una dnssec-policy	38
6.5	Firmado automático de zona	39
6.6	Verificación de la firma	40
6.7	Publicación del DS en el padre	41
6.8	Rollovers: panorama general	42
6.9	Resumen	42
7	05 - Monitoreo y troubleshooting	44
7.1	Qué monitorear, y cada cuánto	44
7.2	Árbol de diagnóstico rápido	44
7.3	Herramientas	45
7.4	El hueco entre el lab y un dominio real	47
7.5	Escenario 1: firma vencida	47
7.6	Escenario 2: DS que no coincide con la DNSKEY (simulado)	49
7.7	Escenario 3: secundario desincronizado	50
7.8	Tabla de referencia: otros síntomas comunes	51
7.9	Resumen	51
8	06 - Arquitectura con hidden signer	52
8.1	Motivación: separar la firma de la publicación	52
8.2	Ventajas	52
8.3	Componentes y flujo	54
8.4	Migrar <code>example.com</code> a esta arquitectura	55
8.4.1	1. Archivo de zona: sale <code>ns1</code> , entra <code>ns3</code> , y el SOA apunta al signer	55
8.4.2	2. El signer: firma, pero no atiende consultas públicas	56
8.4.3	3. Los servidores públicos: <code>ns2</code> sin cambios, <code>ns3</code> nuevo	56
8.4.4	4. Actualizar NS y glue en el padre	57
8.4.5	5. Verificar	57

8.5	Buenas prácticas	58
8.6	Resumen	58
9	07 - DNSSEC avanzado	60
9.1	NSEC3 en profundidad	60
9.1.1	nsec3param: iterations, salt, opt-out	60
9.1.2	Verificación	61
9.2	Mecánica completa de rollovers	62
9.2.1	Dos capas, no una	62
9.2.2	ZSK: estrategia pre-publish	63
9.2.3	KSK: estrategia double-KSK	63
9.3	Inspeccionar y operar rollovers en vivo	64
9.3.1	dnssec-settime: la línea de tiempo	64
9.3.2	rndc dnssec -status: la vista operativa	64
9.3.3	Forzar y confirmar un rollover a mano	65
9.4	Algorithm rollover	66
9.5	CDS/CDNSKEY	67
9.6	Multi-signer (RFC 8901, Model 2)	68
9.7	Resumen	68

1 Introducción

Este es un breve curso de DNS y DNSSEC orientado a operadores de redes pequeños y medianos, incluyendo ISPs, IXPs, operadores de TLDs pequeños y universidades entre otros.

El curso asume que el lector:

- Está familiarizado con la operación de servidores basados en Unix, en particular Linux Ubuntu.
- Domina los conceptos de alta disponibilidad, sockets, servicios en Unix.
- Está familiarizado con los conceptos más básicos de DNS. Si bien el texto que sigue intenta ser explicativo de todos los conceptos, no es un curso de DNS básico.
- Está familiarizado con los conceptos de cifrado, cifrado de clave pública y algoritmo de hash. Si bien se incluyen pequeñas explicaciones sobre estas ideas son mas bien a título de recordatorio y este texto no es un curso de criptografía.

El espíritu detrás de este texto es el promover la experimentación. No hay ninguna cantidad de lectura que pueda suplir el experimentar el funcionamiento del DNS de manera directa. El uso de máquinas virtuales y containers hace mucho más accesible el poder experimentar con todos los elementos de una instalación realista de DNS.

Carlos Martinez, carlos@lacnic.net.

Montevideo, 7 de agosto de 2026.

2 Temario

2.1 1. Introducción a DNSSEC

- ¿Qué es DNS y por qué necesita seguridad adicional?
- Ataques que DNSSEC mitiga: cache poisoning, spoofing, ataques on-path
- Qué problemas NO resuelve DNSSEC (confidencialidad, DDoS, etc.)
- Cadena de confianza: de la raíz a la zona hoja
- Conceptos clave: firma, clave pública/privada, validación

2.2 2. Los registros de DNSSEC

- RRSIG: firma de un conjunto de registros (RRset)
- DNSKEY: clave pública de la zona (ZSK y KSK)
- DS: enlace de confianza hacia el padre (delegation signer)
- NSEC: prueba de no-existencia (denegación autenticada)
- Mención breve de NSEC3 (hashing de nombres, cuándo se usa) — sin profundizar
- RRSIG sobre cada tipo de registro, incluyendo sobre NSEC/NSEC3

2.3 3. Zonas autoritativas con BIND

- Instalando BIND 9 en Ubuntu
- Repaso del layout de la configuración de bind9 en Ubuntu
- Crear una zona autoritativa primaria
- Verificar el funcionamiento con dig/drill
- Configurar una zona autoritativa secundaria
 - permitir axfr/ixfr de forma segura y controlada
- Consideraciones de seguridad
 - prevenir la recursion no deseada
 - prevenir las respuestas tomadas de cache no deseadas
 - prevenir las transferencias de zona no deseadas
- Comentarios sobre configurar los registros NS en la zona padre

2.4 4. Firmando una zona con BIND y KASP

- Repaso de conceptos: KSK vs ZSK, por qué existe el rollover (panorama general, sin mecánica interna — eso va en el módulo 7)
- Introducción a KASP (Key and Signing Policy) en BIND
- Migrar el primario del módulo 3 a firmado inline (zona pasa a ser dinámica/gestionada por named)
- Configuración de una dnssec-policy en named.conf (algoritmo ECDSAP256SHA256, NSEC — consistente con el módulo 2; NSEC3 queda para el módulo 7)
- Firmado automático de zona (inline-signing / auto-dnssec)
- Verificación de la firma: dig, delv, dnssec-verify
- Publicación del DS en el padre (registrador / TLD)
- Menció de que KASP gestiona los rollovers automáticamente — detalle completo en el módulo 7

2.5 5. Monitoreo y troubleshooting

- ¿Cuales son los problemas mas comunes que nos pueden surgir con una zona firmada?
- Que parametros monitorear de una zona firmada, cada cuanto tiempo
- Herramientas comunes para monitorear una zona firmada
- Diagnostico de problemas
- Resolución de problemas más comunes

2.6 6. Arquitectura con hidden signer

- Motivación: separar la firma de la publicación
- Ventajas: superficie de ataque reducida, aislamiento de claves privadas, disponibilidad
- Componentes: hidden master/signer, servidores públicos secundarios
- Flujo: zona sin firmar → hidden signer → zona firmada → servidores públicos (AXFR/IXFR)
- Implementación con BIND: hidden signer configurado con KASP + notify/also-notify hacia los públicos, servidores públicos como secundarios ocultando el hidden signer
- Buenas prácticas: control de acceso, monitoreo de expiración de firmas, backups de claves

2.7 7. DNSSEC avanzado

- NSEC3: hashing de nombres, iteraciones (RFC 9276: iterations=0, sin salt), opt-out — cuándo y por qué preferirlo sobre NSEC

- Mecánica completa de rollovers gestionados por KASP: línea de tiempo de una clave (generada → publicada → activa → retirada → eliminada) y la máquina de estados por tipo de registro que la sostiene (hidden → rumoured → omnipresent → unretentive, para DNSKEY/DS/RRSIG); estrategia pre-publish (ZSK, doble DNSKEY) vs. double-KSK (KSK, doble firma sobre el DNSKEY RRset + doble DS durante la transición con el padre)
- Inspeccionar y operar rollovers en vivo: `dnssec-settime`, `rndc dnssec -status / -rollover / -checkds`
- Algorithm rollover: cuándo migrar de algoritmo y cómo `dnssec-policy` gestiona la firma dual durante la transición
- CDS/CDNSKEY: publicación automática para simplificar (donde el registrador lo soporte) la sincronización del DS con el padre
- Multi-signer (RFC 8901, Model 2): motivación, soporte en BIND vía `dnssec-policy`, coordinación de DNSKEY/DS entre proveedores independientes

3 01 - Introducción a DNSSEC

3.1 Repaso breve de DNS

DNS traduce nombres (`example.com`) a datos (direcciones IP, entre otras cosas). Los datos se organizan en **zonas**, cada zona tiene un archivo con **RRsets** (conjuntos de registros del mismo nombre y tipo).

Ejemplo simplificado de una zona `example.com` sin firmar:

```
example.com.      3600  IN  SOA  ns1.example.com. admin.example.com. (
                        2026071001 ; serial
                        3600      ; refresh
                        600       ; retry
                        1209600   ; expire
                        3600 )    ; minimum
example.com.      3600  IN  NS   ns1.example.com.
example.com.      3600  IN  NS   ns2.example.com.
example.com.      3600  IN  A    93.184.216.34
www.example.com.  3600  IN  CNAME example.com.
```

Las zonas se **delegan**: la zona raíz (.) delega `com.` a los servidores del TLD, y `com.` delega `example.com.` a los servidores del dominio. Un resolver sigue esa cadena de delegaciones, servidor por servidor, hasta obtener la respuesta.



El problema: nada en este mecanismo prueba que una respuesta sea auténtica. Un resolver acepta lo que recibe.

3.2 Fundamentos de criptografía

Antes de seguir, tres conceptos que DNSSEC usa todo el tiempo.

3.2.1 Criptografía

Conjunto de técnicas matemáticas para proteger información: garantizar que solo cierta parte pueda leerla (confidencialidad), o que se pueda comprobar su origen e integridad (autenticidad). DNSSEC usa la segunda rama — no cifra datos, los **firma**.

3.2.2 Criptografía de clave pública

En criptografía simétrica hay una sola clave, compartida entre las partes, que sirve tanto para cifrar como para descifrar. Eso no funciona en DNS: no hay forma de compartir un secreto con millones de resolvers sin exponerlo.

La criptografía de clave pública (asimétrica) usa un **par de claves** matemáticamente relacionadas:

- una **clave privada**, secreta, que se usa para firmar,
- una **clave pública**, distribuida libremente, que se usa para verificar.

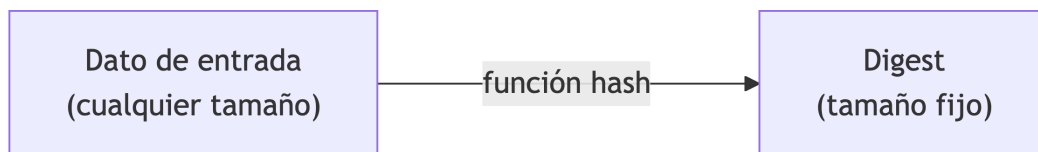
Lo que se firma con una clave privada, solo se verifica con su clave pública correspondiente. Nadie necesita conocer la clave privada para confiar en lo firmado — solo la clave pública, que puede publicarse sin problema. Esto es lo que hace viable el modelo: la clave pública de una zona se publica en la propia zona (registro DNSKEY, ver [módulo 2](#)).

3.2.3 Hashes (digest / resúmenes)

Un **hash** (o *digest*) es el resultado de aplicar una función matemática a un dato de cualquier tamaño, produciendo una salida de tamaño fijo. Las funciones de hash criptográficas usadas en DNSSEC (SHA-256, SHA-1 en desuso) tienen tres propiedades clave:

- **Determinismo**: el mismo dato siempre produce el mismo hash.
- **Efecto avalancha**: cambiar un solo bit del dato de entrada cambia por completo el hash resultante — no hay forma de predecir el hash a partir de una modificación pequeña.
- **Resistencia a colisiones y a preimagen**: es computacionalmente inviable encontrar dos datos distintos con el mismo hash, o reconstruir el dato original a partir de su hash.

Esto hace que un hash sirva como “huella digital” de un dato: no se puede usar para reconstruirlo, pero cualquier alteración del original es detectable comparando hashes.



Un ejemplo concreto. Este haiku:

Sobre la rama
una hoja de otoño
cae en silencio

tiene estos hashes:

Algoritmo	Hash
MD5	c5f12d2a6fe419e912c9e57296792872
SHA-1	13c3b99562aa840a4b95aa080e8add1fcfb02609

Si le agregamos un solo punto al final (“...silencio.”) — un cambio mínimo, invisible a simple vista si no se compara con cuidado — los hashes son completamente distintos:

Algoritmo	Hash
MD5	27ae6b2b4c1808e8df03f92a06b31a43
SHA-1	2e4b10306f75266216133fbfd0cd503021ac6c42

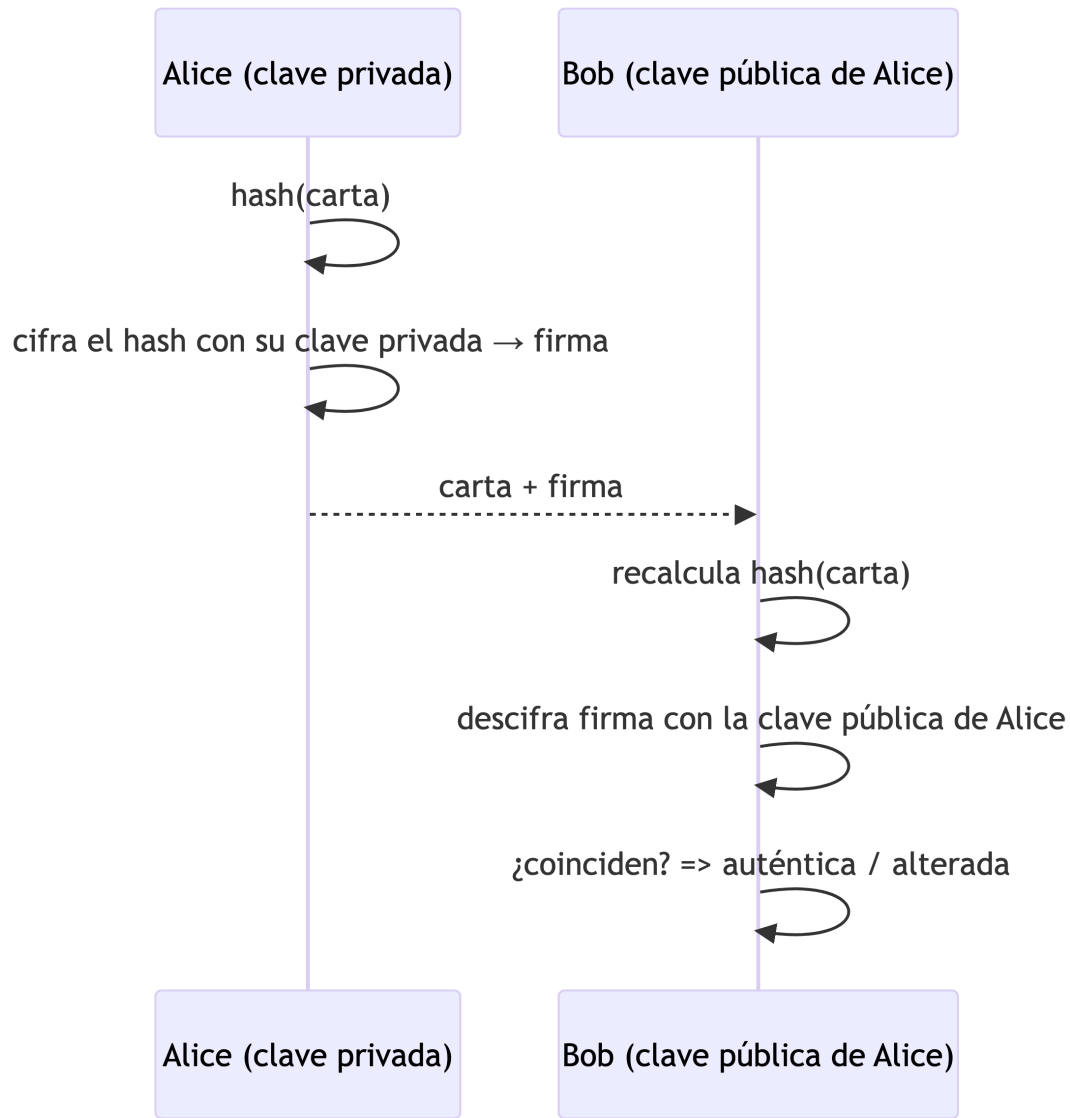
Ningún patrón conecta ambos hashes — ese es el efecto avalancha en acción. (Nota al margen: MD5 y SHA-1 aparecen aquí solo con fines ilustrativos; ambos están **rotos** para uso criptográfico — se conocen ataques de colisión práctica. DNSSEC usa SHA-256 o superior.)

DNSSEC usa hashes en dos lugares distintos: para reducir un RRset a un resumen antes de firmarlo (ver [Firma digital](#) abajo), y para el registro DS, que es literalmente el hash de una clave DNSKEY publicado en la zona padre — así el padre certifica la clave del hijo sin tener que republicarla entera (ver [módulo 2](#)).

3.2.4 Firma digital

Firmar un dato no es lo mismo que cifrarlo. El ejemplo clásico: Alice quiere enviarle una carta a Bob, y Bob quiere poder comprobar que realmente la escribió Alice y que nadie la alteró en el camino.

1. Alice calcula un **hash** de su carta (un resumen de tamaño fijo; cualquier cambio en el texto cambia completamente el hash).
2. Alice cifra ese hash con su **clave privada** — el resultado es la firma, que envía junto con la carta.
3. Bob, con la **clave pública** de Alice, descifra la firma y recalcula el hash de la carta recibida. Si ambos hashes coinciden, la carta es auténtica y no fue alterada; si no coinciden, algo cambió — o no la firmó Alice.



Este es exactamente el mecanismo que DNSSEC aplica a los RRsets de una zona, empaquetado en un registro RRSIG. Con esta base, firmar una zona y validar una respuesta ya tienen un sentido concreto.

3.3 ¿Qué es DNSSEC?

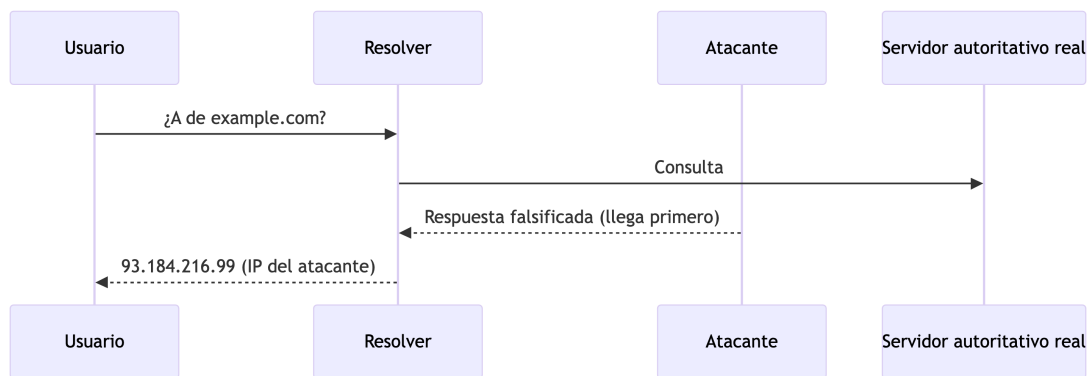
DNSSEC (Domain Name System Security Extensions) es un conjunto de extensiones al protocolo DNS que aplica firma digital a las respuestas. No cifra nada — cualquiera puede seguir leyendo el tráfico DNS — pero permite a un resolver **validar** que una respuesta:

- fue generada por el dueño legítimo de la zona (autenticidad), y
- no fue modificada en tránsito (integridad).

Esto se logra firmando los RRsets con la clave privada de la zona y publicando la clave pública correspondiente en la propia zona, más un enlace hacia el padre que certifica esa clave. El detalle de los registros involucrados se ve en el [módulo 2](#).

3.4 ¿Qué problemas resuelve?

El caso clásico es el **cache poisoning** (ataque Kaminsky, 2008): un atacante que logra adivinar o inyectar una respuesta falsa antes de que llegue la legítima puede envenenar la caché de un resolver, redirigiendo a los usuarios a servidores maliciosos sin que noten nada.



Con DNSSEC, el resolver valida la firma de la respuesta contra la cadena de confianza. Si la respuesta del atacante no está firmada correctamente (no tiene la clave privada de **example.com**), la validación falla y el resolver la descarta.

En términos generales, DNSSEC protege contra cualquier escenario donde un tercero (on-path o fuera de camino) intenta **suplantar o alterar** respuestas DNS: spoofing, envenenamiento de caché, ataques de intermediario sobre resoluciones no cifradas.

3.5 ¿Qué NO resuelve DNSSEC?

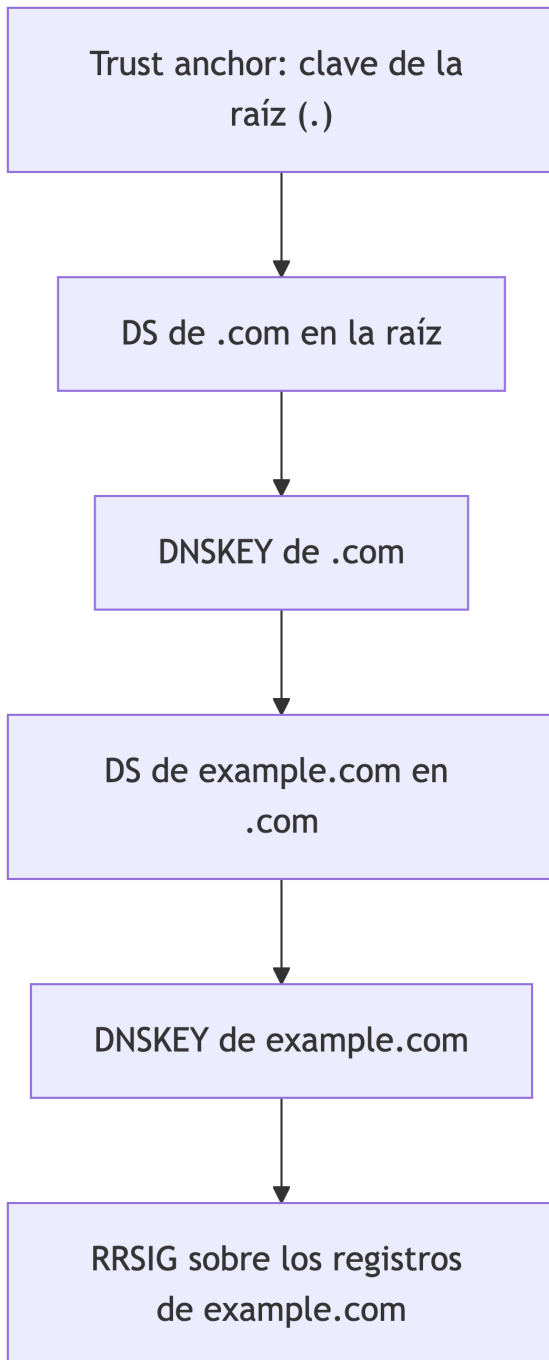
Es igual de importante entender los límites:

- **No es una solución para brindar confidencialidad.** Las consultas y respuestas siguen viajando en texto claro; cualquiera que observe el tráfico ve qué se consulta. Para eso existen DoT/DoH/DoQ, que son ortogonales a DNSSEC.
- **No protege contra DDoS.** Un servidor autoritativo o un resolver saturado sigue siendo vulnerable a ataques de denegación de servicio.

- **No garantiza disponibilidad.** Si la firma expira o hay un error de configuración, el efecto es que las respuestas se vuelven *inválidas*, no que dejen de existir — un resolver validador simplemente las rechaza (posible caída del servicio si no se opera bien).
- **No valida el contenido en sí**, solo que el contenido no fue alterado desde que el dueño de la zona lo firmó. Si el dueño de la zona publica un registro incorrecto, DNSSEC lo firma igual.

3.6 Cadena de confianza

La validación funciona porque existe una cadena continua desde un **ancla de confianza** (la clave pública de la zona raíz, distribuida fuera de banda) hasta la zona que se está consultando. Cada eslabón certifica al siguiente:



Si en algún eslabón la cadena se rompe (falta un DS, una firma expiró, una clave no coincide), la validación falla para toda la zona y todo lo que cuelga debajo de ella.

3.7 Conceptos clave

Término	Significado breve
Hash	Resumen (<i>digest/hash</i>) de tamaño fijo de un dato; cambia por completo si el dato cambia
Validador	Resolver que verifica firmas antes de aceptar una respuesta
RRset	Conjunto de registros del mismo nombre y tipo (lo que se firma como unidad)

KSK, ZSK y los tipos de registro específicos de DNSSEC se cubren en detalle en el próximo módulo.

3.8 Resumen

- La firma digital con criptografía de clave pública es la base técnica de DNSSEC: clave privada firma, clave pública verifica.
- DNSSEC agrega autenticidad e integridad a las respuestas DNS, no confidencialidad ni disponibilidad.
- Resuelve ataques de suplantación/envenenamiento de caché mediante firmas verificables.
- Depende de una cadena de confianza ininterrumpida desde la raíz hasta la zona consultada.
- Siguiente: [02-Registros.md](#) — los registros concretos que hacen esto posible.

4 02 - Los registros de DNSSEC

Nota sobre los ejemplos: `example.com` no está firmado en la vida real, así que para ver registros DNSSEC reales usamos **isc.org** (Internet Systems Consortium, mantenedores de BIND), uno de los dominios pioneros en firmar su zona. Las consultas se hicieron el 10/07/2026 — si las repetís, los valores van a diferir (las firmas y claves rotan).

DNSSEC agrega cuatro tipos de registro nuevos a una zona: **DNSKEY**, **RRSIG**, **DS** y **NSEC**. Cada uno cumple un rol distinto en la cadena de confianza vista en el [módulo 1](#).

4.1 DNSKEY — la clave pública de la zona

Contiene la clave pública de la zona. Se publica en la propia zona para que cualquier validador pueda obtenerla.

```
$ dig +dnssec +multiline DNSKEY isc.org
```

```
isc.org. 405 IN DNSKEY 256 3 13 (
    1CS+VQcRn4lGTK+b3wDjV00hFDx4DV7s3Q1Fwxuq9ahd
    255FRny4f4vdZOMMMxpbRH5Zhwoh/706IV0v9Jwj1A==
    ) ; ZSK; alg = ECDSAP256SHA256 ; key id = 27566
isc.org. 405 IN DNSKEY 257 3 13 (
    zEoOfseNFDM+E8spu7RR2Ar/GzFqAehe4yapWLiv6McI
    UF6xmI5GcIQ3+uLAizS2cNWHt6EArVj8ogjtrRXfw==
    ) ; KSK; alg = ECDSAP256SHA256 ; key id = 7250
```

isc.org publica **dos** DNSKEY: una ZSK (Zone Signing Key) y una KSK (Key Signing Key). El propio `dig` anota el rol y el `key id` de cada una entre paréntesis — de dónde sale esa distinción y por qué se usan dos claves en vez de una se explica en detalle en el [módulo 4](#), cuando toque generarlas.

4.2 RRSIG — la firma sobre un RRset

Es la firma digital (la del [módulo 1](#)) empaquetada como registro DNS. Cada RRset firmado tiene su propio RRSIG.

```
$ dig +dnssec +multiline A isc.org

isc.org. 300 IN A 151.101.2.217
isc.org. 300 IN A 151.101.66.217
isc.org. 300 IN A 151.101.130.217
isc.org. 300 IN A 151.101.194.217
isc.org. 300 IN RRSIG A 13 2 300 (
    20260719192313 20260705182333 27566 isc.org.
    kh8odndDXlsLHt7vumqWyXTdf/HbiiPfxqPli0Dg6+QZ
    D9nxorNCXuA2E0I807oC8tJ5FzL0wfgbtmUtkDAqyg== )
```

Lo importante del RRSIG, sin entrar en el formato binario completo:

- 13 — algoritmo de firma (se detalla en el módulo 4).
- 20260719192313 / 20260705182333 — vencimiento e inicio de validez de la firma. Fuera de ese rango, la firma es inválida aunque los datos no hayan cambiado.
- 27566 — key id de la DNSKEY que firmó este RRset (coincide con la ZSK que vimos arriba).
- `isc.org.` — nombre de la zona firmante.

Fijate que la KSK (7250) **no** firmó este registro A — lo firmó la ZSK (27566). Si consultás el DNSKEY RRset vas a ver lo contrario:

```
$ dig +dnssec +multiline DNSKEY isc.org | grep RRSIG -A2

isc.org. 405 IN RRSIG DNSKEY 13 2 3600 (
    20260720060847 20260706053216 7250 isc.org.
    ...
```

Ese 7250 es la KSK. Patrón general: **la KSK firma únicamente el DNSKEY RRset; la ZSK firma todo lo demás** (A, SOA, NSEC, etc.). El porqué de esta separación de roles se ve en el módulo 4.

4.3 DS — el enlace hacia el padre

El DS (“Delegation Signer”) no vive en la zona misma, sino en la zona **padre**. Es un hash de la KSK, y es lo que permite extender la cadena de confianza de un eslabón al siguiente.

```
$ dig +dnssec +multiline DS isc.org

isc.org. 3600 IN DS 7250 13 2 (
    A30B3F78B6DDE9A4A9A2AD0C805518B4F49EC62E7D3F
    4531D33DE697CDA01CB2 )
isc.org. 3600 IN RRSIG DS 8 2 3600 (
    20260730154208 20260709144208 13950 org.
    ...
```

Notá tres cosas:

- El 7250 inicial es el key id de la KSK de **isc.org** — el DS apunta a esa clave específica.
- El DS de **isc.org** está firmado por la clave 13950, que pertenece a la zona **org.**, no a **isc.org**. Esto es literalmente el eslabón de la cadena: el padre certifica al hijo.
- Un validador calcula el hash de la KSK que recibió de **isc.org** y lo compara contra este DS. Si coinciden, la KSK es de confianza; si no, la validación falla.

4.4 NSEC — negar que algo no existe, de forma autenticada

DNS necesita poder responder “esto no existe” (NXDOMAIN). Sin DNSSEC, esa respuesta negativa no está firmada — un atacante podría falsificarla para ocultar un nombre real. NSEC resuelve esto firmando la **ausencia**.

```
$ dig +dnssec +multiline A doesnotexist.isc.org

;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN

docs.isc.org. 3600 IN NSEC dommel.isc.org. A RRSIG NSEC
docs.isc.org. 3600 IN RRSIG NSEC 13 3 3600 ( ... )
isc.org.      3600 IN NSEC _acme-challenge.isc.org. A NS SOA MX TXT AAAA ...
isc.org.      3600 IN RRSIG NSEC 13 2 3600 ( ... )
```

Un NSEC dice “no existe ningún nombre entre este registro y el siguiente, en orden alfabético”. Acá **docs.isc.org.** apunta a **dommel.isc.org.** — como **doesnotexist.isc.org**

cae alfabéticamente entre ambos, y el NSEC (firmado) lo confirma, el resolver puede probar criptográficamente que el nombre no existe, no solo confiar en un NXDOMAIN sin firmar.

El NSEC también lista, al final, qué tipos de registro *sí* existen para ese nombre (A NS SOA MX TXT ... RRSIG NSEC) — útil para probar autenticadamente ausencia de un tipo puntual (ej. “este nombre existe pero no tiene AAAA”).

NSEC3 es una variante que usa nombres *hasheados* en vez de nombres en claro, para evitar que alguien recorra el NSEC de la zona y enumere todos los nombres existentes (zone walking). La lógica de fondo es la misma; lo vas a encontrar en zonas grandes o sensibles a enumeración. No entramos en su formato acá.

4.5 Resumen

Registro	Vive en	Firma/protege
DNSKEY	La propia zona	Publica las claves públicas (ZSK/KSK)
RRSIG	La propia zona	La firma de un RRset puntual
DS	La zona padre	El hash de la KSK del hijo — extiende la cadena de confianza
NSEC	La propia zona	Prueba autenticada de que un nombre/tipo no existe

- Todo RRset firmado tiene su RRSIG correspondiente; el DNSKEY RRset lo firma la KSK, el resto lo firma la ZSK.
- El DS es el único registro DNSSEC que no vive en la zona que protege — vive un nivel arriba.
- NSEC (no NSEC3) fue lo que vimos en el ejemplo; ambos resuelven el mismo problema de denegación autenticada.
- Siguiente: [03-Zonas-BIND.md](#) — levantar un servidor BIND y publicar una zona autoritativa (sin firmar todavía).

5 03 - Zonas autoritativas con BIND

Hasta ahora vimos DNSSEC en abstracto: qué problema resuelve y qué registros usa. Antes de firmar nada hace falta un servidor autoritativo funcionando. Este módulo levanta ese servidor con BIND 9 sobre Ubuntu — sin firma todavía, eso es el [módulo 4](#). Vamos a publicar la misma zona `example.com` que usamos como ejemplo en el [módulo 1](#), ahora en un servidor real.

5.1 Instalando BIND 9 en Ubuntu

```
$ sudo apt update
$ sudo apt install bind9 bind9utils bind9-dnsutils
```

- `bind9` — el propio servidor (`named`).
- `bind9utils` — herramientas de administración: `rndc`, `named-checkzone`, `named-checkconf`, `dnssec-*` (estas últimas se usan recién en el módulo 4).
- `bind9-dnsutils` — `dig`, `nslookup`, `delv`.

Verificación básica:

```
$ named -v
BIND 9.18.28 (Extended Support Version) <id:...>

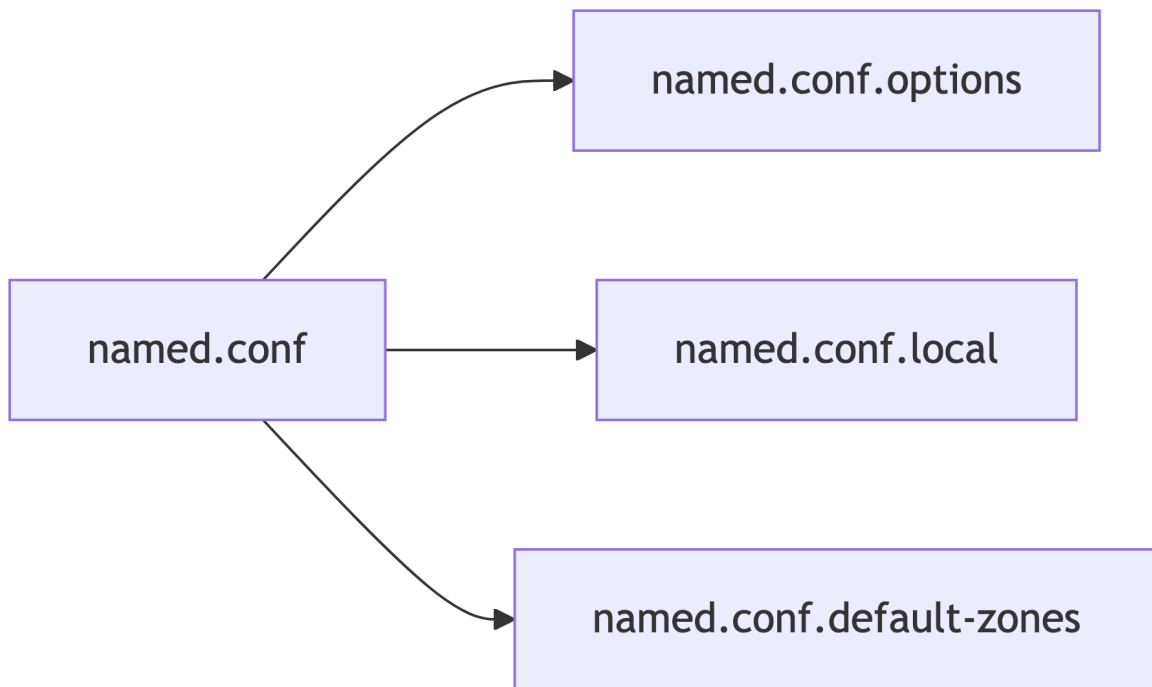
$ systemctl status bind9
bind9.service - BIND Domain Name Server
   Active: active (running)
```

Un detalle propio de Ubuntu: BIND corre bajo AppArmor con un perfil que restringe a qué directorios puede acceder `named` (por defecto `/etc/bind`, `/var/cache/bind`, `/var/lib/bind`). Si más adelante movés zonas a otra ruta, hay que actualizar el perfil (`/etc/apparmor.d/usr.sbin.named`) o vas a ver errores de permiso que no aparecen en `named.conf` sino en el log de AppArmor (`journalctl -k | grep apparmor`).

5.2 Repaso del layout de configuración en Ubuntu

Ubuntu separa la configuración de BIND en varios archivos, todos bajo `/etc/bind/`:

Archivo	Contenido
<code>named.conf</code>	Solo tres líneas <code>include</code> hacia los archivos de abajo — no se edita directamente
<code>named.conf.options</code>	Opciones globales del servidor: recursión, listen-on, forwarders, etc.
<code>named.conf.local</code>	Donde se declaran las zonas propias (lo que vamos a editar en este módulo)
<code>named.conf.default-zones</code>	Zonas que Ubuntu preconfigura: <code>localhost</code> , reversa de loopback, <code>root.hints</code>



Los datos viven aparte de la configuración:

- `/var/cache/bind/` — directorio de trabajo por defecto (`directory` en `named.conf.options`). Ahí van los archivos de zona, journals (`.jnl`) y las zonas transferidas de un primario.
- `/etc/bind/rndc.key` — clave usada por `rndc` (la herramienta de control remoto de `named`) para autenticarse contra el servidor. Detalle de configuración y errores comunes, a continuación.

5.3 Configurando rndc

`rndc` es el canal por el que se le habla a `named` en caliente (`reload`, `retransfer`, `freeze`, etc.) — no es un simple wrapper de línea de comandos, es un cliente que se conecta por TCP (puerto 953 por defecto) a un canal de control separado del propio DNS, autenticado con una clave compartida en vez de usuario/contraseña. Vale la pena mirarlo de cerca antes de depender de él, porque en la práctica es de lo primero que falla en una instalación nueva.

Qué trae Ubuntu por defecto. El paquete `bind9` genera la clave automáticamente al instalarse (`/etc/bind/rndc.key`), y `named.conf` la incluye:

```
$ cat /etc/bind/named.conf
include "/etc/bind/rndc.key";
include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";
```

Si no declarás un bloque `controls` explícito en `named.conf.options`, `named` abre igual, por comportamiento por defecto, un canal de control en `127.0.0.1:953` usando esa clave. Por eso `rndc reload` local suele andar apenas instalado el paquete — pero “andar” acá tiene requisitos que no siempre están cubiertos:

```
$ ls -l /etc/bind/rndc.key
-rw-r----- 1 bind bind 77 ago  4 10:00 /etc/bind/rndc.key
```

El archivo es legible solo por `root` y por el grupo `bind`. Si corrés `rndc` como un usuario normal sin `sudo`, va a fallar — y el mensaje de error no dice “permiso denegado”, dice algo más genérico:

```
$ rndc reload
rndc: connection to remote host closed
This may indicate that
* the remote server is using an older/higher version of rndc protocol,
* this host is not authorized to connect,
* the clocks are not synchronized, or
* the connection key is invalid.
```

Ese mismo mensaje aparece por varias causas distintas, en orden de probabilidad cuando algo que “debería andar solo” no anda:

1. **Falta de permisos para leer la clave** — el caso más común. Solución: `sudo rndc reload`, o agregar el usuario al grupo `bind` (`sudo usermod -aG bind $USER`, requiere volver a loguear).
2. **Reloj desincronizado** — `rndc` firma el mensaje con timestamp; un desfase grande entre cliente y servidor invalida la firma aunque la clave sea correcta. Si lo anterior no era el problema, revisar `timedatectl`.
3. **La clave de `named.conf` y la de quien corre `rndc` no coinciden** — típicamente porque alguien regeneró `/etc/bind/rndc.key` a mano, o editó el bloque `controls/key`, y quedó desactualizado en un lado.

Regenerar la clave (por ejemplo, para pasar del HMAC-MD5 que trae por defecto en instalaciones viejas a algo más fuerte):

```
$ sudo rndc-confgen -a -A hmac-sha256
$ sudo systemctl restart bind9
```

`rndc-confgen -a` sobrescribe `/etc/bind/rndc.key` directamente — no hace falta tocar `named.conf.options` a mano si vas a seguir controlando el servidor solo desde `localhost`.

Habilitar `rndc` desde otra máquina. Este es el caso que efectivamente hace falta para el `rndc` retransfer de la próxima sección, si el primario y el secundario están en hosts distintos: el canal de control, por default, solo escucha en loopback. Nada externo puede hablarle a `named` aunque tenga la clave correcta — hay que declarar `controls` explícitamente en `named.conf.options` del servidor que se quiere controlar:

```
controls {
    inet 192.0.2.2 port 953
        allow { 203.0.113.10; } keys { "rndc-key"; };
};
```

Y en la máquina desde la que se va a correr `rndc` (un host de administración, o el propio secundario apuntando a sí mismo) hace falta un `/etc/bind/rndc.conf` — que Ubuntu no genera por default, hay que crearlo:

```
# /etc/bind/rndc.conf
key "rndc-key" {
    algorithm hmac-sha256;
    secret "misma-clave-base64-que-en-rndc.key==";
};

options {
    default-server 192.0.2.2;
```

```
    default-key "rndc-key";  
};
```

El secreto tiene que ser textualmente el mismo que en el `/etc/bind/rndc.key` del servidor controlado. Con esto, `rndc -s 192.0.2.2 reload example.com` ya no depende de un `rndc.key` local — usa lo declarado en `rndc.conf`.

5.4 Crear una zona autoritativa primaria

En `named.conf.local`:

```
zone "example.com" {  
    type primary;  
    file "/etc/bind/db.example.com";  
};
```

(En versiones viejas de BIND — y en gran parte de la documentación dando vueltas — vas a ver `type master`; en vez de `type primary`;. Son sinónimos; `primary/secondary` es la terminología actual.)

Archivo de zona `/etc/bind/db.example.com`, coherente con el que vimos sin firmar en el módulo 1:

```
$TTL 3600  
example.com.      IN  SOA  ns1.example.com. admin.example.com. (  
                                2026080401 ; serial  
                                3600      ; refresh  
                                600       ; retry  
                                1209600   ; expire  
                                3600 )    ; minimum  
example.com.      IN  NS   ns1.example.com.  
example.com.      IN  NS   ns2.example.com.  
ns1.example.com.  IN  A     192.0.2.1  
ns2.example.com.  IN  A     192.0.2.2  
example.com.      IN  A     93.184.216.34  
www.example.com.  IN  CNAME example.com.
```

Antes de recargar, validar sintaxis:

```
$ named-checkzone example.com /etc/bind/db.example.com
zone example.com/IN: loaded serial 2026080401
OK
```

```
$ named-checkconf
```

`named-checkconf` sin salida es éxito. Si algo está mal, avisa con el archivo y número de línea exacto — conviene correrlo siempre antes de recargar, cuesta un segundo y evita tirar el servicio con una zona rota.

Recargar y confirmar que `named` la tomó:

```
$ sudo rndc reload
zone reload successful
```

```
$ journalctl -u named --since "1 min ago"
... zone example.com/IN: loaded serial 2026080401
```

5.5 Verificar el funcionamiento con dig/drill

```
$ dig @127.0.0.1 example.com A +short
93.184.216.34
```

```
$ dig @127.0.0.1 example.com SOA +short
ns1.example.com. admin.example.com. 2026080401 3600 600 1209600 3600
```

`drill` (de `ldnsutils`, `apt install ldnsutils`) es una alternativa más compacta a `dig`, útil más adelante para inspeccionar DNSSEC:

```
$ drill @127.0.0.1 example.com A
;; ->>HEADER<<- opcode: QUERY, rcode: NOERROR, id: ...
;; ANSWER SECTION:
example.com.      3600      IN      A       93.184.216.34
```

Si cualquiera de los dos falla, revisar en este orden: `named-checkzone` (sintaxis), `systemctl status bind9` (¿está corriendo?), `journalctl -u named` (mensaje de error concreto), y por último si el puerto 53 está escuchando (`ss -ltnu | grep :53`).

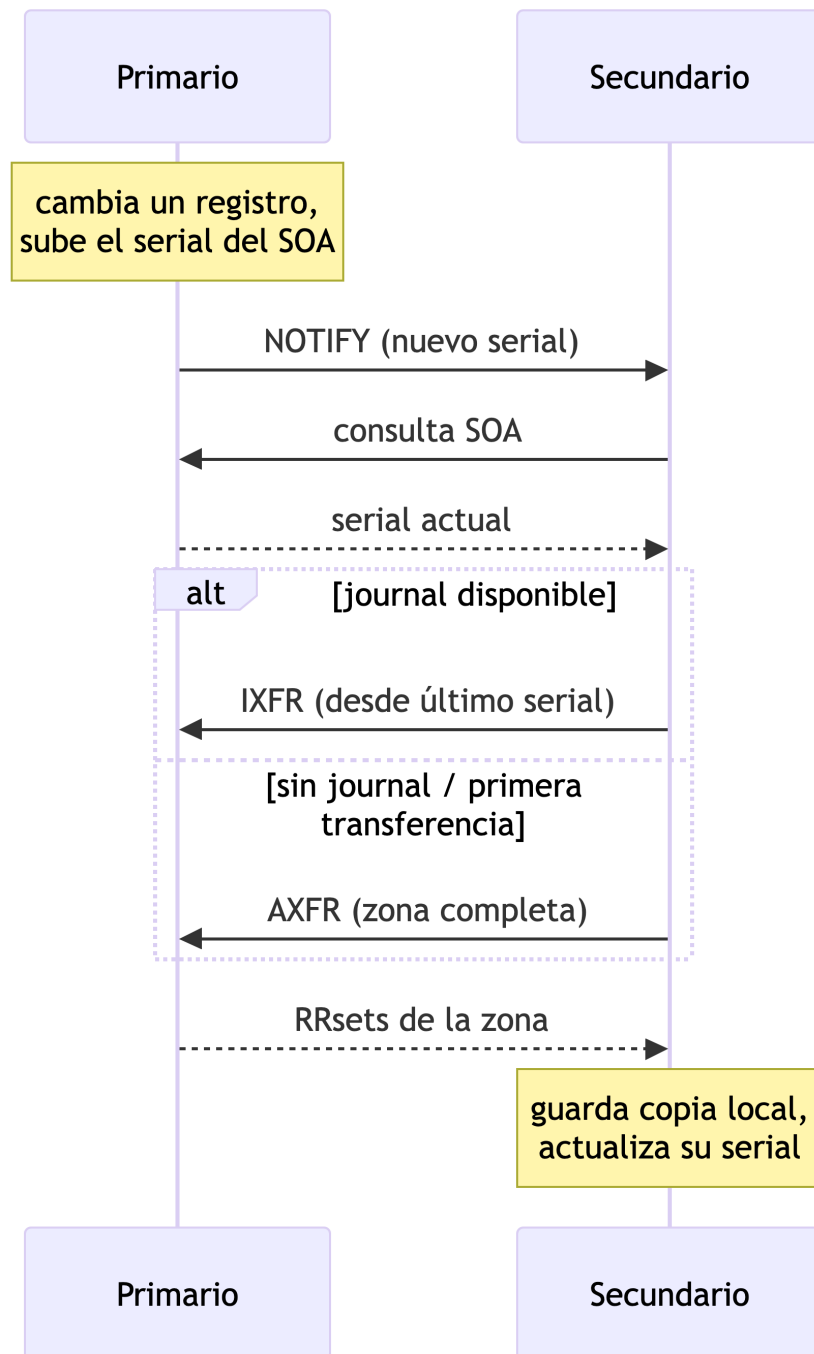
5.6 Configurar una zona autoritativa secundaria

Por qué usar un secundario. Un único servidor autoritativo es un punto único de falla — si se cae o queda inalcanzable, nadie puede resolver la zona, sin importar cuán bien esté firmada o configurada. Las buenas prácticas (RFC 2182) piden al menos dos servidores autoritativos, en redes y, preferentemente, ubicaciones distintas, para que la caída de uno no tire la zona entera. Como beneficio adicional, servidores dispersos geográficamente acortan la latencia para resolvers cercanos y reparten la carga de consultas.

Cómo se mantienen sincronizados. Un secundario no tiene un archivo de zona editado a mano — lo obtiene y actualiza por **transferencia de zona** desde el primario, disparada por dos mecanismos que suelen combinarse:

- **Polling por SOA:** cada **refresh** segundos (el valor del SOA que vimos en el módulo 1), el secundario le pregunta al primario su serial actual. Si es mayor al que tiene guardado, inicia una transferencia.
- **NOTIFY:** el primario avisa proactivamente al secundario apenas cambia el serial, en vez de que este último tenga que esperar el próximo poll — lo configuramos más abajo con **also-notify/notify yes**, y acorta la propagación de minutos/horas a segundos.

El rol de AXFR. La transferencia en sí ocurre mediante **AXFR** (“Authoritative Transfer”), un tipo de consulta DNS distinto de una consulta normal (A, MX, etc.): en vez de UDP, va por **TCP** — porque la respuesta puede ser arbitrariamente grande, toda la zona en un solo intercambio, algo que UDP no soporta bien — y en vez de devolver un RRset puntual, devuelve **la zona completa**, como una secuencia de mensajes DNS que empieza y termina con el registro SOA (eso le indica al receptor dónde arranca y dónde termina la transferencia). Es el mecanismo que arma la copia inicial del secundario, y al que se recurre si en algún momento pierde sincronía — por ejemplo, si el journal de IXFR (la variante incremental, que solo transfiere los cambios desde el último serial conocido) no está disponible.



En el `named.conf.local` del secundario:

```
zone "example.com" {
```

```

type secondary;
primaries { 192.0.2.1; }; // IP del servidor primario
file "/var/cache/bind/example.com.secondary";
};

```

(`primaries` es el nombre actual del bloque; en configuraciones antiguas aparece como `masters`, mismo significado.)

Los NS tienen que reflejar la topología real. Fijate que 192.0.2.2 — la IP que le vamos a asignar al secundario — es la misma que ya le dimos a `ns2.example.com` en el archivo de zona del primario, al principio de este módulo. Eso no es casualidad: es el punto que hace que la alta disponibilidad primario/secundario funcione de verdad. Tener la zona sincronizada en el secundario no alcanza si nadie lo va a consultar — para eso, **tanto el primario como el secundario necesitan su propio registro NS, declarado en dos lugares:**

1. **En el archivo de zona** — ya lo tenemos: NS `ns1.example.com` y NS `ns2.example.com`, cada uno con su A correspondiente (192.0.2.1 y 192.0.2.2).
2. **En la zona padre** (registrador/TLD, cubierto al final de este módulo) — tiene que delegar a los dos, no solo al primario. Si el padre delega solo a `ns1`, `ns2` puede estar perfectamente sincronizado y aun así ser irrelevante: los resolvers nunca lo van a descubrir, porque nunca lo van a preguntar por él.

Sin los dos NS coherentes en ambos lugares, un “secundario” deja de ser alta disponibilidad real y pasa a ser, en el mejor de los casos, un respaldo manual que hay que promover a mano si el primario cae.

Con esto solo, la transferencia va a fallar — el primario, por defecto, no deja transferir la zona a nadie. Eso se resuelve en la sección siguiente.

Una vez habilitada la transferencia, confirmar en el secundario:

```

$ sudo rndc retransfer example.com
$ ls -la /var/cache/bind/example.com.secondary
$ dig @<ip-secundario> example.com SOA +short

```

Para ver el AXFR en crudo, tal cual lo recibe el secundario (una vez habilitado el permiso en la sección siguiente):

```

$ dig @192.0.2.1 example.com AXFR

```

El serial debe coincidir con el del primario. Si no aparece el archivo, revisar `journalctl -u named` en el secundario — ahí BIND registra explícitamente por qué rechazó o no pudo completar la transferencia.

5.6.1 Permitir AXFR/IXFR de forma segura y controlada

En el **primario**, dentro del bloque de la zona (o globalmente en `named.conf.options` si aplica a todas):

```
acl "secundarios" {
    192.0.2.2;    // IP del secundario
};

zone "example.com" {
    type primary;
    file "/etc/bind/db.example.com";
    allow-transfer { key transfer-key; };
    also-notify { 192.0.2.2; };
    notify yes;
};
```

`also-notify + notify yes` hacen que el primario avise al secundario apenas cambia el serial, en vez de esperar a que el secundario pregunte por su cuenta según el **refresh** del SOA.

Restringir por IP (`allow-transfer { secundarios; };`) es un mínimo, pero una IP se puede falsificar. Para autenticar de verdad, TSIG (Transaction Signature — una clave simétrica compartida entre primario y secundario):

```
$ tsig-keygen transfer-key
key "transfer-key" {
    algorithm hmac-sha256;
    secret "base64...==";
};
```

Ese bloque se copia igual en ambos servidores (`named.conf.options` o un archivo incluido), y luego se referencia en `allow-transfer { key transfer-key; };` en el primario, y en el bloque `primaries` del secundario:

```
primaries { 192.0.2.1 key transfer-key; };
```

La elección entre AXFR e IXFR (vistas arriba) es automática: BIND prefiere IXFR si hay `journal (.jnl)` disponible, y solo recurre a AXFR completo si no lo hay — no hace falta configurar nada extra para que el secundario prefiera la incremental, es una negociación automática entre las dos partes. El `dig ... AXFR` manual sigue siendo útil para depurar o inspeccionar la zona completa, aunque el flujo normal en producción use IXFR la mayor parte del tiempo.

5.7 Consideraciones de seguridad

5.7.1 Prevenir la recursión no deseada

Un servidor autoritativo no debería resolver consultas recursivas para terceros. Mezclar los dos roles en la misma instancia es la fuente más común de servidores mal configurados (open resolvers), y agranda innecesariamente la superficie de ataque del servidor autoritativo. En `named.conf.options`:

```
options {
    recursion no;
    allow-recursion { none; };
    ...
};
```

Regla general: un servidor es **autoritativo** (responde solo por zonas que aloja) o **recursivo/resolver** (resuelve para clientes), nunca las dos cosas a la vez en producción.

5.7.2 Prevenir las respuestas tomadas de caché no deseadas

El cache poisoning (visto en el módulo 1) ataca específicamente a resolvers que cachean respuestas de terceros para reusarlas. Un servidor autoritativo con `recursion no` ya no arma ese tipo de caché — es la primera y principal mitigación. Dos opciones complementarias, para cuando conviene ser explícito:

```
options {
    additional-from-cache no;
    additional-from-auth no;
};
```

Evitan que `named` complete secciones adicionales de una respuesta (glue, por ejemplo) usando datos que no vienen directamente de una zona que el servidor aloja — reduce todavía más la chance de que datos de terceros terminen filtrando en una respuesta autoritativa.

5.7.3 Prevenir las transferencias de zona no deseadas

Por defecto, dejar `allow-transfer` abierto (o sin declarar en versiones viejas de BIND, que a veces default a “cualquiera”) permite que cualquiera descargue la zona completa con un simple `dig axfr`. Dos problemas: expone toda la zona de una (reconocimiento fácil para un atacante) y permite que un tercero se pare como secundario no autorizado.

Buena práctica: negar por defecto a nivel global, permitir solo por excepción a nivel de zona:

```
options {
    allow-transfer { none; };
};

zone "example.com" {
    ...
    allow-transfer { key transfer-key; };
};
```

Combinando ACL/TSIG (sección anterior) con este default-deny, solo los secundarios explícitamente autorizados —y autenticados— pueden transferir.

5.8 Comentarios sobre configurar los NS en la zona padre

Publicar NS `ns1.example.com` y NS `ns2.example.com` en la propia zona no alcanza: la zona **padre** (el registrador o el operador del TLD, según cómo esté delegado el dominio) necesita el mismo juego de NS para delegar correctamente. Dos puntos donde esto suele salir mal:

- **Glue records:** si el nombre del servidor NS está *dentro* de la zona que delega (`ns1.example.com` para la zona `example.com`), el padre necesita publicar también el registro A/AAAA de ese NS — si no, hay una referencia circular (para resolver `ns1.example.com` hace falta preguntarle a `ns1.example.com`). Esto se configura del lado del registrador, normalmente en una sección separada de “glue” o “servidores DNS” del panel de administración.
- **Coherencia de delegación:** el set de NS publicado en el padre y el publicado en la propia zona deben coincidir. Si difieren, es una delegación “lame” — algunos resolvers siguen la del padre, otros terminan confundidos, y el comportamiento puede ser inconsistente según qué servidor conteste primero.

Los cambios de NS en el padre suelen tardar en propagarse (TTL del propio registro NS/glue en el padre, más caché de resolvers ya en tránsito) — no es instantáneo, conviene planificarlo con margen si se está migrando de proveedor de DNS.

5.9 Resumen

Concepto	Qué logra
<code>rndc.key</code> + grupo <code>bind</code>	Autentica el control local de <code>named</code> ; la causa más común de “ <code>rndc</code> no anda” es no poder leer esta clave
<code>controls</code> + <code>rndc.conf</code>	Habilita y autentica el control remoto de <code>named</code> desde otro host
<code>type primary</code> / <code>type secondary</code>	Distingue quién tiene la copia autoritativa original de la zona
<code>allow-transfer</code> + TSIG	Restringe y autentica quién puede copiar la zona completa
<code>also-notify</code> / <code>notify yes</code>	El secundario se entera de cambios sin esperar el refresh del SOA
<code>recursion no</code>	Evita que el servidor autoritativo actúe como resolver abierto
Glue records en el padre	Rompe la referencia circular cuando el NS vive dentro de la propia zona

- Un servidor autoritativo primario aloja el archivo de zona original; uno o más secundarios mantienen una copia sincronizada por AXFR/IXFR.
- Todo lo configurado acá es texto plano, sin firmar — cualquiera puede seguir modificando una respuesta en tránsito sin que el resolver lo note.
- Siguiente: [04-Firmando-con-BIND-y-KASP.md](#) — firmar esta misma zona con KASP y publicar el DS en el padre.

6 04 - Firmando una zona con BIND y KASP

Con el primario y el secundario de `example.com` del [módulo 3](#) ya funcionando —pero sirviendo la zona en texto plano—, toca firmarla. Este módulo convierte ese mismo primario en un **inline-signer**: sigue siendo el mismo servidor, pero a partir de acá `named` gestiona las claves y produce automáticamente la versión firmada de la zona. El secundario no necesita ningún cambio — sigue transfiriendo por AXFR/IXFR exactamente como en el módulo 3, solo que ahora lo que transfiere ya viene firmado.

6.1 Repaso: KSK vs ZSK, y por qué el rollover

En el [módulo 2](#) vimos, con `isc.org`, que hay dos DNSKEY con roles distintos: la KSK firma únicamente el DNSKEY RRset, la ZSK firma todo lo demás. La razón de separar los roles:

- El **DS** en el padre apunta al hash de la **KSK**, no de la ZSK. Rotar la ZSK es un asunto puramente interno de la zona — no requiere tocar nada en el padre. Rotar la KSK sí implica actualizar el DS en el padre, un paso manual fuera del control de la zona misma.
- Por eso, en la práctica, la ZSK rota con más frecuencia (menor costo operativo por rotación) y la KSK se mantiene más tiempo estable (cada rotación cuesta una coordinación externa).

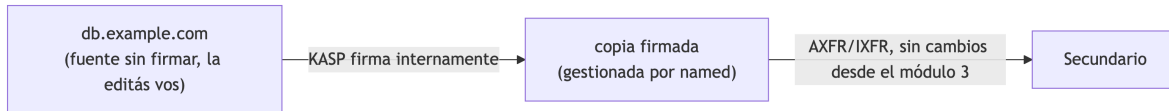
El **rollover** en sí —reemplazar una clave por otra sin romper la validación mientras el cambio se propaga— es lo que motiva la automatización que viene a continuación. La mecánica completa (estados de una clave, períodos de doble firma, cómo se ve un rollover en curso) se cubre en detalle en el módulo 7; acá alcanza con entender que existe y por qué.

6.2 Qué es KASP

KASP (Key and Signing Policy) es el mecanismo con el que BIND automatiza todo el ciclo de vida de las claves y la firma: generación, publicación, uso, rotación y retiro — sin que haga falta correr `dnssec-keygen` y `dnssec-signzone` a mano cada vez, que era el flujo de trabajo antes de que KASP se integrara al propio `named` (BIND 9.16). Una `dnssec-policy` es, en esencia, un perfil reutilizable: algoritmo, tiempos de vida de cada clave, y varios parámetros de la firma, todo declarado una vez y aplicado a una o más zonas.

6.3 Migrar el primario a firmado inline

Hasta el módulo 3, `/etc/bind/db.example.com` era el archivo que editás a mano y que `named` cargaba tal cual. Con **inline-signing**, ese archivo pasa a ser la **fuentes sin firmar**: la seguís editando vos (agregar un registro, cambiar una IP), pero `named` ya no la sirve directamente. En su lugar, mantiene una copia interna firmada — separada, gestionada por él — y es esa copia la que efectivamente se responde en las consultas y se transfiere al secundario.



Esta separación entre “fuente sin firmar” y “copia firmada” —hoy interna, dentro del mismo proceso `named`— es exactamente el concepto que el [módulo 6](#) lleva un paso más allá, separándolas en dos servidores físicos distintos.

Antes de activar la política hace falta un directorio para las claves, con permisos correctos para el usuario `bind` (mismo tipo de detalle de permisos que vimos con AppArmor en el módulo 3 — si usás una ruta fuera de `/etc/bind` o `/var/cache/bind`, hay que actualizar el perfil):

```
$ sudo mkdir -p /etc/bind/keys/example.com
$ sudo chown bind:bind /etc/bind/keys/example.com
```

6.4 Configurando una dnssec-policy

En `named.conf.options` (o un archivo separado incluido desde ahí), se declara la política:

```
dnssec-policy "operadores" {
    keys {
        ksk lifetime unlimited algorithm ecdsap256sha256;
        zsk lifetime P90D algorithm ecdsap256sha256;
    };

    dnskey-ttl PT1H;
    publish-safety PT1H;
    retire-safety PT1H;
    signatures-refresh P5D;
    signatures-validity P2W;
    signatures-validity-dnskey P2W;
    max-zone-ttl P1D;
    purge-keys P90D;
};
```

Algunas notas sobre esta política de ejemplo:

- **Algoritmo `ecdsap256sha256`** — el mismo algoritmo 13 que vimos en los RRSIG de `isc.org` en el módulo 2. Vas a ver el mismo número en el `dig` más abajo.
- **`ksk lifetime unlimited`** — no rota automáticamente; el rollover de KSK, cuando se necesite, se dispara a mano (tiene sentido: cada rollover de KSK implica coordinar el DS en el padre, no es algo para dejar en piloto automático).
- **`zsk lifetime P90D`** — rota cada 90 días sin intervención (formato de duración tipo ISO 8601: `P90D` = 90 días, `PT1H` = 1 hora).
- **Sin `nsec3param`** — al no declararlo, la política usa **NSEC**, lo mismo que vimos en el módulo 2. NSEC3 es una opción explícita (`nsec3param ...`) que se cubre en el módulo 7.

Y en la zona, dentro de `named.conf.local`:

```
zone "example.com" {
    type primary;
    file "/etc/bind/db.example.com";
    key-directory "/etc/bind/keys/example.com";
    dnssec-policy "operadores";
    inline-signing yes;

    allow-transfer { key transfer-key; };
    also-notify { 192.0.2.2; };
    notify yes;
};
```

Todo lo demás (`allow-transfer`, `also-notify`, la ACL/TSIG) es exactamente lo que configuramos en el módulo 3 — no cambia nada del lado de la transferencia de zona.

6.5 Firmado automático de zona

Validar y recargar como siempre:

```
$ named-checkconf
$ sudo rndc reload example.com
zone example.com/IN: reloaded
```

En el log, `named` va a mostrar que generó las claves y empezó a firmar:

```
$ journalctl -u named --since "1 min ago"
... zone example.com/IN: reconfiguring zone keys
... zone example.com/IN: DNSKEY (Kexample.com.+013+41230) is now published
... zone example.com/IN: DNSKEY (Kexample.com.+013+54915) is now published
... zone example.com/IN: signing zone: done
```

Las claves quedan en el directorio que declaramos:

```
$ ls /etc/bind/keys/example.com/
Kexample.com.+013+41230.key    Kexample.com.+013+41230.private  (ZSK, key id 41230)
Kexample.com.+013+54915.key    Kexample.com.+013+54915.private  (KSK, key id 54915)
```

+013+ en el nombre del archivo es justamente el algoritmo (13 = ECDSA_P256SHA256) — el mismo número que veníamos rastreando desde el módulo 2. Los key id (41230, 54915) son arbitrarios, generados al crear cada clave — no se eligen a mano.

Para ver el estado de la firma en cualquier momento sin ir al log:

```
$ rndc signing -list example.com
Signing with key 013+41230
Signing with key 013+54915
```

6.6 Verificación de la firma

Con `dig`, ahora `example.com` responde con RRSIG y DNSKEY, igual que vimos con `isc.org` en el módulo 2 — y con el mismo patrón: la KSK (54915) firma el DNSKEY RRset, la ZSK (41230) firma todo lo demás:

```
$ dig @127.0.0.1 example.com DNSKEY +dnssec +multiline
example.com. 3600 IN DNSKEY 256 3 13 ( ...base64... ) ; ZSK; key id = 41230
example.com. 3600 IN DNSKEY 257 3 13 ( ...base64... ) ; KSK; key id = 54915
example.com. 3600 IN RRSIG DNSKEY 13 2 3600 ( ...20260805... 54915 example.com. ... )
```

```
$ dig @127.0.0.1 example.com A +dnssec +short
93.184.216.34
A 13 2 3600 20260819 20260805 41230 example.com. ...
```

`delv` valida la cadena de confianza de punta a punta, no solo que la firma exista:


```
$ delv @127.0.0.1 example.com A
;; validating example.com/A: no valid signature found...
;; resolution failed: insecure
93.184.216.34
```

Ese **insecure** es **esperado en este punto** — no es un error. `delv` (y cualquier validador real) no tiene forma de confiar en la KSK de `example.com` todavía, porque el DS que la certificaría no está publicado en el padre. La zona está firmada y es internamente consistente, pero la cadena de confianza del módulo 1 todavía tiene un eslabón roto — falta el próximo paso.

`dnssec-verify` chequea la señal completa sin depender del padre — sirve para confirmar que la firma es correcta antes de publicar nada:

```
$ dnssec-verify -o example.com /var/cache/bind/example.com.signed
Loading zone 'example.com' from file '/var/cache/bind/example.com.signed'
Verifying the zone using the following algorithms: ECDSAP256SHA256.
Zone fully signed:
Algorithm: ECDSAP256SHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                                ZSKs: 1 active, 0 stand-by, 0 revoked
```

(El nombre exacto del archivo firmado en disco puede variar según la instalación; `rndc zonestatus example.com` muestra la ruta configurada si no es la esperada.)

6.7 Publicación del DS en el padre

El DS es un hash de la KSK — se genera a partir del archivo de clave pública de la KSK, no de la zona firmada:

```
$ dnssec-dsfromkey /etc/bind/keys/example.com/Kexample.com.+013+54915.key
example.com. IN DS 54915 13 2 A94C3B1F... (SHA-256)
```

Ese registro (key tag, algoritmo, tipo de digest, hash) es lo que se sube al **padre** — el registrador o el operador del TLD, en el mismo lugar donde se gestionan los NS y el glue vistos en el módulo 3. El flujo:

1. Confirmar que la zona ya está sirviendo RRSIG/DNSKEY válidos (paso anterior) — **nunca publicar el DS antes de esto**. Si el DS está en el padre pero la zona no firma correctamente, cualquier resolver validador empieza a rechazar toda la zona.
2. Cargar el DS en el panel del registrador/TLD.
3. Esperar propagación (TTL del DS en el padre + caché de resolvers, igual que con los NS en el módulo 3).

4. Volver a correr `delv` — con el DS ya propagado, el resultado debería pasar de `insecure` a validado:

```
$ delv @127.0.0.1 example.com A
; fully validated
93.184.216.34
```

Con esto, la cadena de confianza del módulo 1 (raíz → TLD → `example.com`) queda completa.

6.8 Rollovers: panorama general

KASP ya está rotando la ZSK cada 90 días según la política, sin que haga falta ningún paso manual — el rollover completo (publicar la clave nueva, empezar a firmar con ella, retirar la vieja) ocurre solo, con superposición suficiente para que no haya ventana donde una firma sea inválida.

La KSK es distinta: como su rollover implica actualizar el DS en el padre, y KASP no tiene acceso al panel del registrador, ese paso siempre requiere intervención humana — es la única parte del ciclo que no se puede dejar en automático. Si se rota la KSK sin actualizar el DS a tiempo, la validación se rompe para toda la zona.

Los estados internos de una clave durante un rollover, los tiempos de seguridad entre cada estado, y cómo observar un rollover en curso con `dnssec-settime/rndc dnssec -status`, se ven en detalle en el [módulo 7](#).

6.9 Resumen

Concepto	Qué logra
<code>dnssec-policy</code>	Perfil reutilizable: algoritmo, tiempos de vida de KSK/ZSK, parámetros de firma
<code>inline-signing yes</code>	El archivo de zona pasa a ser fuente sin firmar; <code>named</code> mantiene la copia firmada por separado
<code>dnssec-dsfromkey</code>	Genera el registro DS a partir de la KSK, para publicar en el padre
DS en el padre	Cierra la cadena de confianza — sin esto, la zona firma pero nadie la valida
Rollover de ZSK vs KSK	El de ZSK es automático e interno; el de KSK requiere actualizar el DS a mano

- El mismo primario del módulo 3 ahora firma la zona; el secundario sigue funcionando sin cambios.
- Firmar la zona no alcanza por sí solo — sin el DS publicado en el padre, un validador real la sigue viendo como **insecure**.
- Siguiente: [05-Monitoreo-Troubleshooting.md](#) — qué vigilar en una zona firmada y cómo diagnosticar cuando algo falla.

7 05 - Monitoreo y troubleshooting

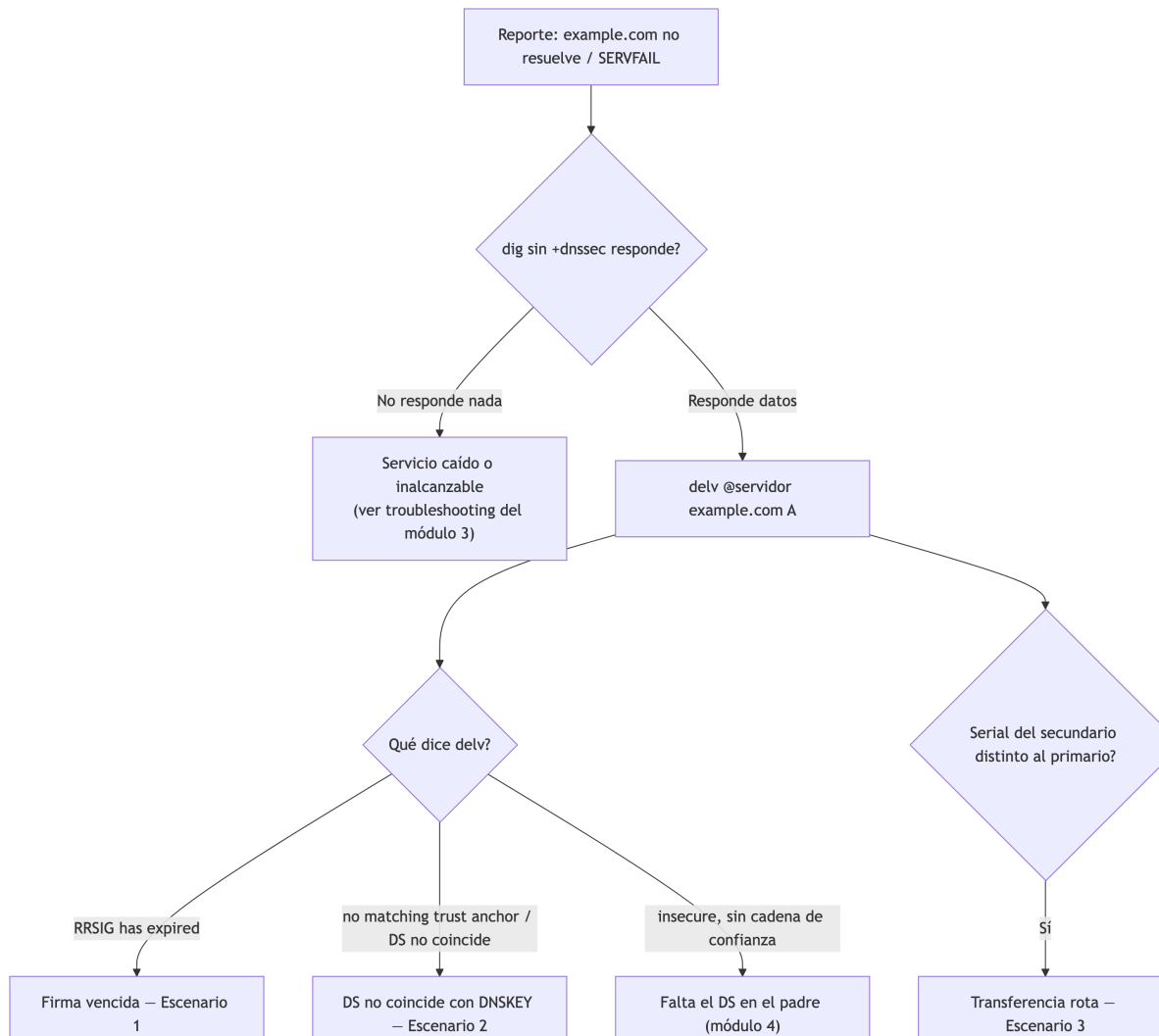
Con `example.com` firmada y funcionando desde el [módulo 4](#), toca la pregunta operativa: ¿qué hay que vigilar para que siga funcionando, y qué hacer cuando algo se rompe? La mayoría de los incidentes de DNSSEC en producción no son ataques — son firmas que vencieron sin que nadie se diera cuenta, un DS mal copiado al registrador, o un secundario que quedó desincronizado en silencio. Este módulo cubre los tres, reproduciéndolos deliberadamente sobre el mismo `example.com` de los módulos 3 y 4, más una tabla de referencia para el resto de los síntomas comunes.

7.1 Qué monitorear, y cada cuánto

Parámetro	Cómo chequearlo	Frecuencia sugerida
Tiempo restante antes de que venza el RRSIG más próximo	<code>dig +dnssec</code> sobre el RRset, o un script (más abajo)	Diario
DS del padre vs. DNSKEY vigente de la zona	<code>dig DS</code> al padre + <code>dnssec-dsfromkey</code> sobre la KSK activa	Tras cualquier rollover de KSK; rutina semanal igual
Serial del primario vs. cada secundario	<code>dig SOA</code> a cada servidor	Cada pocos minutos, o alertar directamente sobre fallos de NOTIFY/transferencia en el log
NS y glue del padre vs. la propia zona	<code>dig NS</code> al padre + <code>dig NS</code> a la zona	Tras cualquier cambio de NS; rutina mensual
Si hay un rollover en curso	<code>rndc signing -list</code> (mecánica completa en el módulo 7)	Según el calendario de la <code>dnssec-policy</code>

7.2 Árbol de diagnóstico rápido

Ante un reporte de “esto no resuelve” o `SERVFAIL`, el orden que ahorra más tiempo:



7.3 Herramientas

Lo que ya venimos usando desde los módulos 3 y 4 sigue siendo la base:

Herramienta	Para qué
<code>dig +dnssec</code>	Ver RRSIG/DNSKEY tal cual los recibe un cliente
<code>delv</code>	Validar la cadena de confianza completa, igual que un resolver validador
<code>rndc zonestatus / rndc signing -list</code>	Estado de firmado de una zona sin ir al log

Herramienta	Para qué
<code>named-checkzone</code> / <code>dnssec-verify</code>	Validar sintaxis y firma antes de confiar en que algo “debería andar”

Un script simple de alerta, para no depender de acordarse de correr `dig` a mano:

```
#!/bin/bash
# chequea-rrsig.sh - alerta si una firma vence en menos de N días
ZONA="example.com"
UMBRAL_DIAS=3

VENCE=$(dig +dnssec @127.0.0.1 "$ZONA" A | awk '/^example.com.*RRSIG/ {print $9; exit}')
VENCE_EPOCH=$(date -d "${VENCE:0:8} ${VENCE:8:2}:${VENCE:10:2}:${VENCE:12:2}" +%s)
AHORA_EPOCH=$(date +%s)
DIAS_RESTANTES=$(( (VENCE_EPOCH - AHORA_EPOCH) / 86400 ))

if [ "$DIAS_RESTANTES" -lt "$UMBRAL_DIAS" ]; then
    echo "ALERTA: RRSIG de $ZONA vence en $DIAS_RESTANTES día(s)" | logger -t dnssec-check
fi
```

Corrido por `cron` una vez al día, esto cubre el escenario más común de todos: nadie se entera de que la firma automática dejó de renovarse hasta que un resolver empieza a rechazar la zona.

Herramientas externas (fuera del CLI local, útiles sobre todo contra un dominio realmente delegado):

- **DNSViz** (dnsviz.net, o el paquete `dnsviz` por línea de comandos) — grafica la cadena de confianza completa de una zona, marcando en rojo exactamente qué eslabón falla. Es probablemente la herramienta más rápida para entender un problema de validación que no es obvio con `delv`.
- **Zonemaster** (zonemaster.net) — batería de chequeos más amplia, no solo DNSSEC: delegación, glue, consistencia entre servidores, etc. Bueno como chequeo periódico de salud general de una zona delegada.
- **Monitoreo continuo** — `named` expone estadísticas por `statistics-channels` (JSON/XML) que un exporter de Prometheus (`bind_exporter`) puede scrapear; Zabbix tiene templates de comunidad para alertar sobre vencimiento de firmas. No entramos en la configuración de ninguno de los dos acá — alcanza con saber que existen si el script de `cron` se queda corto para una operación con muchas zonas.

7.4 El hueco entre el lab y un dominio real

`example.com` no tiene un DS real publicado en ningún padre — no está delegado en el internet real, así que `delv` local es lo más cerca que podemos llegar a “validar como lo haría un resolver”. Contra un dominio de verdad, conviene además chequear con algo que vea la zona desde afuera: DNSViz (arriba) o directamente un resolver público con `+cd` (*checking disabled*, para comparar la respuesta con y sin validación):

```
$ dig @1.1.1.1 example.com A           # como lo ve un resolver validador
$ dig @1.1.1.1 example.com A +cd       # con la validación desactivada, para comparar
```

Si la primera falla (o devuelve `SERVFAIL`) y la segunda funciona, el problema es específicamente de DNSSEC — no de DNS en general. Es el mismo patrón que vamos a reproducir localmente con `delv` en los tres escenarios siguientes.

7.5 Escenario 1: firma vencida

La causa más común en la práctica: `named` estuvo caído (mantenimiento largo, falla de hardware, lo que sea) más tiempo del que dura la validez de las firmas. Una firma tiene fecha de vencimiento absoluta — no le importa si el servidor estuvo vivo o no mientras tanto.

Para reproducir esto en minutos en vez de semanas, achicamos temporalmente la política **solo para este ejercicio**:

```
dnssec-policy "operadores" {
    keys {
        ksk lifetime unlimited algorithm ecdsap256sha256;
        zsk lifetime P90D algorithm ecdsap256sha256;
    };
    signatures-validity PT10M;    // 10 minutos - normalmente P2W, ver módulo 4
    signatures-refresh PT5M;      // normalmente P5D
    dnskey-ttl PT1H;
    ...
};

$ sudo rndc reconfig
$ dig @127.0.0.1 example.com A +dnssec +short | tail -1
A 13 2 3600 20260805143000 20260805132000 41230 example.com. ...
```

(El primer timestamp es el vencimiento — ahora a solo 10 minutos.) Simulamos la caída:

```
$ sudo systemctl stop bind9
$ sleep 720    # 12 minutos - más que la validez de 10
$ sudo systemctl start bind9
```

named vuelve a levantar sirviendo la última zona firmada que tenía en disco — con la firma ya vencida, hasta que su ciclo de mantenimiento la detecte y renueve:

```
$ dig @127.0.0.1 example.com A +dnssec +short | tail -1
A 13 2 3600 20260805143000 20260805132000 41230 example.com. ...    # sigue siendo la vieja, v

$ delv @127.0.0.1 example.com A
;; validating example.com/A: verify failed due to bad signature (keyid=41230): RRSIG has exp
;; validating example.com/A: no valid signature found
;; broken trust chain resolving 'example.com/A/IN'
;; resolution failed: SERVFAIL
```

Este SERVFAIL es distinto del `insecure` que vimos en el módulo 4: acá el DS sí existe y la cadena de confianza está armada — lo que falla es la firma en sí. Un validador real trata esto como **bogus**, no como zona sin firmar, y por eso corta en seco con SERVFAIL en vez de simplemente ignorar DNSSEC.

named se recupera solo apenas nota que el RRSIG venció (el ciclo de mantenimiento de KASP corre también al cargar la zona):

```
$ journalctl -u named --since "1 min ago"
... zone example.com/IN: reconfiguring zone keys
... zone example.com/IN: some RRSIGs have expired; signing zone
... zone example.com/IN: signing zone: done

$ delv @127.0.0.1 example.com A
; fully validated
93.184.216.34
```

No olvidar revertir la política a los valores reales del módulo 4 (`signatures-validity` P2W, `signatures-refresh` P5D) antes de seguir — quedarse con una validez de 10 minutos en cualquier entorno que no sea este ejercicio puntual es buscarse este mismo incidente todas las semanas.

7.6 Escenario 2: DS que no coincide con la DNSKEY (simulado)

Esto reproduce, sin tocar ninguna clave real, el error más común al publicar el DS: copiar mal un carácter al pegarlo en el panel del registrador. El síntoma es idéntico al de un rollover de KSK donde el DS no se actualizó a tiempo (eso se opera en detalle en el módulo 7) — la causa cambia, el diagnóstico es el mismo.

`delv` permite validar contra un ancla de confianza declarada a mano con `-a`, sin depender de un padre real — perfecto para este ejercicio, ya que `example.com` no tiene uno.

Con el DS correcto del módulo 4:

```
$ cat > /tmp/ta-correcto.conf <<EOF
trust-anchors {
    example.com. static-ds 54915 13 2 "A94C3B1F...";
};
EOF

$ delv -a /tmp/ta-correcto.conf @127.0.0.1 example.com A
; fully validated
93.184.216.34
```

Ahora, el mismo archivo pero con el dígito final del hash alterado — simulando el typo al pegarlo en el registrador:

```
$ cat > /tmp/ta-incorrecto.conf <<EOF
trust-anchors {
    example.com. static-ds 54915 13 2 "A94C3B0F...";
};
EOF

$ delv -a /tmp/ta-incorrecto.conf @127.0.0.1 example.com A
;; verify failed: DS does not match DNSKEY
;; no valid trust anchor found
;; resolution failed: SERVFAIL
```

Lo inquietante de este fallo: `dnssec-verify` sobre la zona (módulo 4) sigue diciendo que todo está perfectamente firmado — porque lo está. El problema no está en la zona, está en lo que el padre publica. Es exactamente por eso que conviene, después de cargar el DS en el registrador, volver a chequear con algo que mire desde afuera (DNSViz, o `dig @resolver-publico` como en la sección anterior) en vez de confiar en que “la zona firma bien” es suficiente.

7.7 Escenario 3: secundario desincronizado

Rompemos la transferencia desde el primario, simulando un cambio de ACL o firewall accidental:

```
# en el primario, named.conf.local - le sacamos el acceso al secundario
zone "example.com" {
    ...
    allow-transfer { key transfer-key; };    // dejamos esto, pero...
};
```

```
$ sudo iptables -A INPUT -s 192.0.2.2 -p tcp --dport 53 -j DROP    # simula el firewall/ACL r
```

Hacemos un cambio cualquiera en la zona (sube el serial) y recargamos:

```
$ sudo rndc reload example.com
```

En el secundario, la transferencia falla en silencio — sigue respondiendo consultas con su última copia buena, sin avisar por sí solo que quedó atrás:

```
$ dig @192.0.2.1 example.com SOA +short    # primario
ns1.example.com. admin.example.com. 2026080502 ...
```

```
$ dig @192.0.2.2 example.com SOA +short    # secundario - serial viejo
ns1.example.com. admin.example.com. 2026080401 ...
```

```
$ sudo rndc retransfer example.com    # corrido en el secundario
$ journalctl -u named --since "1 min ago"
... transfer of 'example.com/IN' from 192.0.2.1#53: failed to connect: timed out
```

Diagnóstico: mismo orden que en el módulo 3 (ACL/TSIG del lado del primario, conectividad, logs de ambos lados). La combinación peligrosa con el escenario 1: un secundario desincronizado que sigue sirviendo RRSIGs cada vez más viejos, sin que nadie lo note hasta que esas firmas también vencen — dos fallas independientes que se potencian.

Arreglo y verificación:

```
$ sudo iptables -D INPUT -s 192.0.2.2 -p tcp --dport 53 -j DROP
$ sudo rndc retransfer example.com    # en el secundario
$ dig @192.0.2.2 example.com SOA +short
ns1.example.com. admin.example.com. 2026080502 ...    # ya sincronizado
```

7.8 Tabla de referencia: otros síntomas comunes

Síntoma	Causa probable	Cómo confirmarlo	Solución
Delegación “lame” — algunos resolvers fallan, otros no	NS del padre no coincide con el de la zona	<code>dig NS</code> al padre vs. a la zona (módulo 3)	Corregir NS/glue en el registrador
Validación falla solo para algunos resolvers, no todos	TTL viejo del DS/DNSKEY todavía en caché en tránsito tras una corrección reciente	Repetir la consulta desde varios resolvers, comparar	Esperar el TTL — no es un problema nuevo, es propagación
Zona no vuelve a firmar tras un cambio manual al archivo fuente	Se perdió la asociación <code>dnssec-policy/inline-signing</code> en la zona	<code>named-checkconf</code> , <code>rndc zonestatus example.com</code>	Revisar que la zona siga referenciando la política (módulo 4)
<code>rndc</code> deja de responder justo después de un cambio de hora del sistema	Reloj desincronizado — mismo problema que ya vimos con <code>rndc</code> en el módulo 3, pero también rompe la validación de firmas	<code>timedatectl</code>	Sincronizar con NTP; revisar si además hay RRSIGs afectados
Un algoritmo de firma nuevo no valida en algunos resolvers	El resolver es viejo y no soporta ese algoritmo todavía	<code>dig +dnssec</code> desde ese resolver puntual	Evaluar si conviene esperar antes de migrar de algoritmo en producción

7.9 Resumen

- La mayoría de los incidentes de DNSSEC son operativos, no ataques: firmas que no se renovaron, un DS mal publicado, un secundario que se quedó atrás en silencio.
- `delv` distingue tres resultados que no son lo mismo: `insecure` (no hay DS, módulo 4), `SERVFAIL`/bogus por firma vencida (escenario 1), y `SERVFAIL`/bogus por DS que no coincide con la DNSKEY (escenario 2) — el mensaje exacto de `delv` dice cuál es cuál.
- Una zona puede firmar perfectamente (`dnssec-verify` conforme) y aun así fallar la validación para todo el mundo, si el problema está en lo que el padre publica.
- Un secundario desincronizado no avisa solo — hay que vigilar el serial activamente, no asumir que “si responde, está al día”.
- Siguiente: [06-Arquitectura-Hidden-Signer.md](#) — separar el firmado de la publicación en servidores distintos.

8 06 - Arquitectura con hidden signer

En el [módulo 4](#) señalamos algo que quedó pendiente: dentro del mismo proceso `named`, inline-signing ya separa la **fente sin firmar** de la **copia firmada** — son dos cosas distintas, solo que conviven en el mismo servidor. Este módulo lleva esa misma separación un paso más allá: a dos servidores físicos distintos, donde el que firma **nunca** es alcanzable directamente desde internet.

8.1 Motivación: separar la firma de la publicación

Hasta acá, 192.0.2.1 (el primario del módulo 3, firmando desde el módulo 4) hace dos trabajos a la vez: sostiene las claves privadas y firma la zona, **y** está públicamente delegado como `ns1.example.com`, respondiendo consultas de cualquiera en internet. Eso significa que cualquier vulnerabilidad en el servicio DNS público — un bug de `named`, una consulta malformada, un ataque de denegación de servicio — pone en juego, en el peor caso, al mismo proceso que tiene las claves privadas cargadas en memoria.

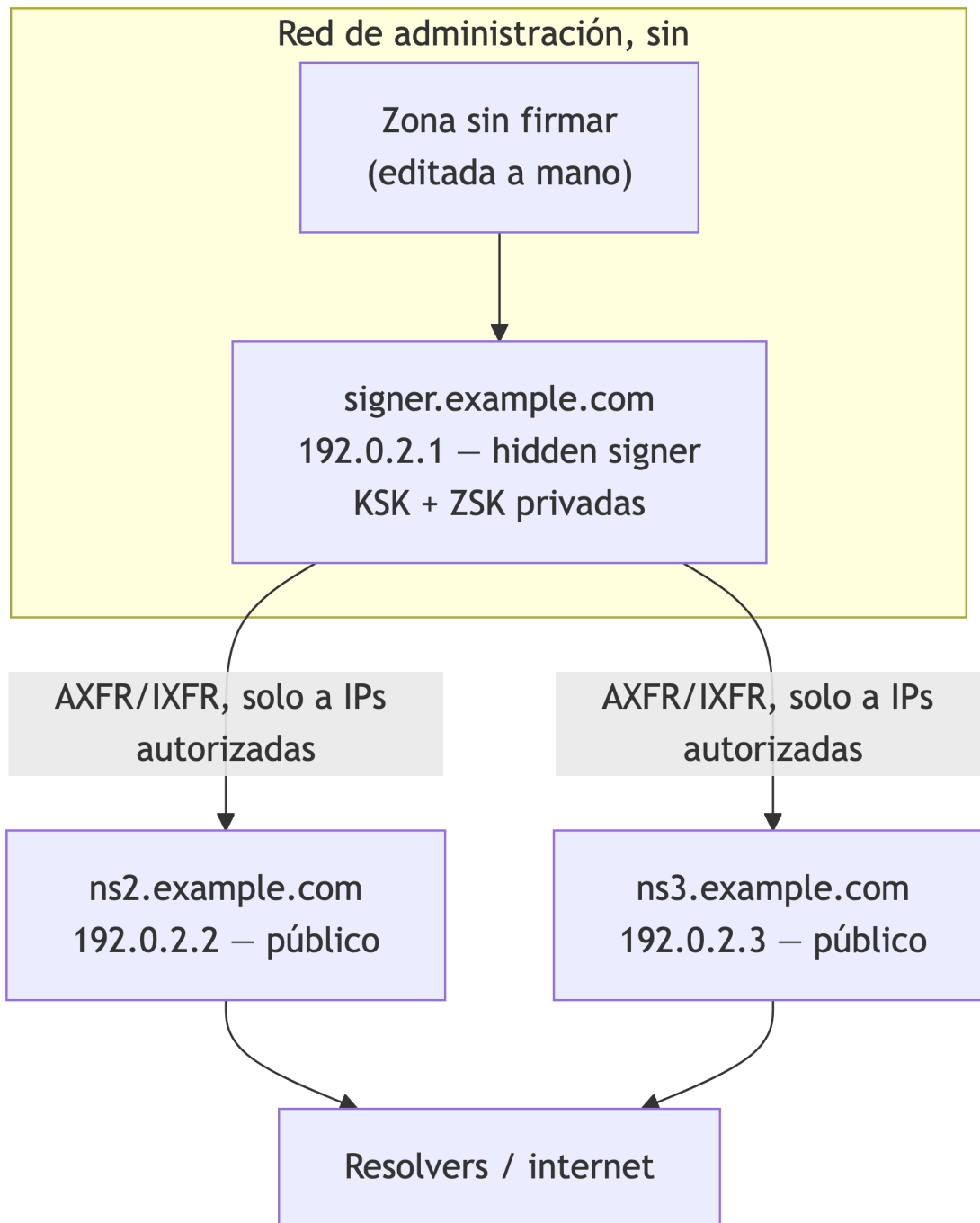
Un **hidden signer** (o *hidden master*) rompe esa combinación: el servidor que firma no publica nada directamente. Solo un puñado de servidores públicos, de confianza explícita, lo consultan por transferencia de zona — y son esos servidores públicos, sin ninguna clave privada, los que efectivamente responden a internet.

8.2 Ventajas

- **Superficie de ataque reducida:** el signer no acepta consultas DNS comunes de nadie. No hay superficie pública para bugs de parseo, amplificación, ni reconocimiento — el único protocolo que expone es la transferencia de zona, y solo hacia IPs específicas y autenticadas.
- **Aislamiento de claves privadas:** comprometer un servidor público (`ns2`, `ns3`) no expone ninguna clave — esas máquinas nunca tuvieron una KSK ni una ZSK, solo la zona ya firmada.
- **Disponibilidad:** los servidores públicos pueden multiplicarse y distribuirse geográficamente sin multiplicar el riesgo de exposición de claves. Si el signer queda temporalmente inalcanzable, los públicos siguen respondiendo con la última zona firmada que tienen —

hasta que las firmas se acerquen a vencer (el mismo escenario que reproducimos en el módulo 5).

8.3 Componentes y flujo



Notá que el signer no tiene ninguna flecha hacia “Resolvers / internet” — esa es exactamente

la propiedad que estamos construyendo.

8.4 Migrar example.com a esta arquitectura

192.0.2.1 deja de llamarse `ns1.example.com` — ese nombre implica “nameserver público”, y ya no lo es. Pasa a ser `signer.example.com`, y sale por completo del NS RRset de la zona. 192.0.2.2 (el `ns2` del módulo 3) sigue exactamente igual — ya era un secundario del 192.0.2.1, y eso no cambia; solo cambia qué es 192.0.2.1 de cara al mundo. Agregamos un tercer servidor, 192.0.2.3 como `ns3.example.com`, para no quedar con un único punto público (la misma razón del módulo 3 para tener más de un servidor autoritativo).

8.4.1 1. Archivo de zona: sale ns1, entra ns3, y el SOA apunta al signer

```
$TTL 3600
example.com.      IN  SOA  signer.example.com. admin.example.com. (
                                2026080601 ; serial
                                3600       ; refresh
                                600        ; retry
                                1209600    ; expire
                                3600 )     ; minimum
example.com.      IN  NS   ns2.example.com.
example.com.      IN  NS   ns3.example.com.
ns2.example.com.  IN  A     192.0.2.2
ns3.example.com.  IN  A     192.0.2.3
example.com.      IN  A     93.184.216.34
www.example.com.  IN  CNAME example.com.
```

Dos cambios respecto al módulo 3, y los dos son intencionales:

- **El MNAME del SOA (`signer.example.com`) no aparece en el NS RRset.** No es un error — el MNAME identifica al servidor donde se originan los cambios de la zona, no tiene por qué ser público, y en una arquitectura hidden-signer casi nunca lo es. Vale la pena dejarlo explícito en la documentación del equipo, porque a primera vista parece una inconsistencia.
- **`ns1.example.com` desaparece.** No se apunta a otra IP, no queda como alias — se borra. Dejar un NS “fantasma” apuntando a un host que ya no responde consultas públicas es peor que no tenerlo: un resolver que todavía lo recuerde va a intentar consultarlo y fallar antes de probar con `ns2/ns3`.

8.4.2 2. El signer: firma, pero no atiende consultas públicas

`named.conf.local` en 192.0.2.1, sobre la base del módulo 4:

```
zone "example.com" {
    type primary;
    file "/etc/bind/db.example.com";
    key-directory "/etc/bind/keys/example.com";
    dnssec-policy "operadores";
    inline-signing yes;

    allow-transfer { key transfer-key; };
    also-notify { 192.0.2.2; 192.0.2.3; };
    notify yes;
};
```

Nuevo respecto al módulo 3/4 — `also-notify` ahora tiene dos destinos en vez de uno, y en `named.conf.options`:

```
options {
    recursion no;
    allow-recursion { none; };
    allow-query { none; };
    ...
};
```

`allow-query { none; }` es la pieza que realmente hace a este servidor “hidden”: en el módulo 3 controlamos quién puede *transferir* la zona completa (`allow-transfer`), pero el servidor seguía respondiendo consultas normales (**dig A**) a cualquiera. Acá se lo negamos incluso eso — el signer no le contesta una consulta DNS común a nadie, ni siquiera por su propia zona. Su única función de red es firmar y transferir a `ns2/ns3`.

8.4.3 3. Los servidores públicos: ns2 sin cambios, ns3 nuevo

`ns2` (192.0.2.2) no necesita ningún cambio de configuración — ya era secundario de 192.0.2.1, y sigue siéndolo; lo único que cambió es qué es 192.0.2.1 puertas afuera. `ns3` se levanta con la misma receta del módulo 3:


```
zone "example.com" {
    type secondary;
    primaries { 192.0.2.1 key transfer-key; };
    file "/var/cache/bind/example.com.secondary";
};
```

Un endurecimiento nuevo acá, del lado de los públicos: restringir de quién aceptan un NOTIFY, para que nadie pueda gatillarles una retransferencia con un NOTIFY falsificado:

```
options {
    allow-notify { 192.0.2.1; };
};
```

8.4.4 4. Actualizar NS y glue en el padre

Mismo procedimiento del módulo 3, ahora con contenido real que cambia: quitar el glue de `ns1` (192.0.2.1), agregar el de `ns3` (192.0.2.3), mantener el de `ns2`. El orden importa para no dejar una ventana de delegación rota:

1. Agregar `ns3` (glue + NS) al padre **antes** de sacar `ns1` — durante la transición, el padre delega a los tres.
2. Confirmar que `ns2` y `ns3` ya están sirviendo la zona firmada correctamente (`dig/delv` contra ambos).
3. Recién ahí, quitar `ns1` del padre.
4. Esperar el TTL del NS/glue viejo antes de dar la migración por terminada — algunos resolvers van a seguir intentando `ns1` hasta que ese TTL expire.

Saltear el paso 1 (sacar `ns1` antes de tener `ns3` funcionando) deja a la zona con un único servidor público durante la transición — exactamente el escenario de punto único de falla que el módulo 3 buscaba evitar.

8.4.5 5. Verificar

```
$ dig @192.0.2.2 example.com A +dnssec +short      # ns2, responde firmado
$ dig @192.0.2.3 example.com A +dnssec +short      # ns3, responde firmado
$ dig @192.0.2.1 example.com A +short              # el signer - sin respuesta
;; connection timed out; no servers could be reached
```

Ese último timeout **es el resultado esperado** — si 192.0.2.1 contesta esa consulta, `allow-query { none; }` no está funcionando como debería.

8.5 Buenas prácticas

Control de acceso, más allá de lo ya configurado arriba: el firewall (o el security group, según dónde viva esto) debería negar por completo el tráfico entrante al signer salvo desde las IPs de `ns2` y `ns3` — `allow-query/allow-transfer` en `named.conf` son la segunda capa, no la única. Si alguien puede llegar por red al puerto 53 del signer aunque `named` se niegue a responder, sigue habiendo superficie (el propio proceso `named` recibiendo paquetes). El acceso administrativo (SSH, `rndc` remoto si aplica) merece el mismo tratamiento: red de gestión separada, no la misma que usan los clientes DNS.

Monitoreo: nada nuevo que agregar acá — todo lo del módulo 5 (vencimiento de firmas, sincronía de seriales, DS vs. DNSKEY) aplica igual, salvo que ahora hay que vigilarlo contra `ns2/ns3`, no contra el signer, porque el signer es precisamente el servidor al que no se le hacen consultas normales.

Backups de claves:

- Lo que hay que respaldar no es solo `Kexample.com.+013+*.key/.private` — KASP también mantiene archivos de estado (metadata de cada clave: cuándo se publicó, cuándo entra en uso, próximos eventos) en el mismo directorio. Un backup que solo copia las claves y no ese estado puede reconstruir la criptografía pero pierde el historial que KASP usa para decidir el próximo paso de un rollover.
- Cifrado en reposo (el `key-directory` completo, no archivo por archivo) y al menos una copia fuera del propio signer — si el disco del signer se pierde sin backup, se pierde la zona firmable, no solo el servicio.
- La severidad de perder una clave no es la misma para las dos: perder la ZSK sin backup es molesto pero recuperable — KASP genera una nueva y rota, sin depender de nadie más. Perder la **KSK** sin backup es mucho peor: como el DS en el padre certifica esa KSK específica, no hay forma de “generar una nueva” sin repetir el procedimiento completo de publicación del DS del módulo 4 — con la zona efectivamente insegura mientras tanto.
- Probar la restauración, no solo hacerla. Un backup que nunca se restauró es una suposición, no una garantía.

8.6 Resumen

Concepto	Qué logra
<code>allow-query { none; }</code> en el signer	El signer no responde consultas DNS comunes — su única función de red es firmar y transferir
<code>also-notify</code> a ambos públicos	El signer avisa a los dos servidores públicos, no solo a uno

Concepto	Qué logra
<code>allow-notify</code> en los públicos	Evita que un NOTIFY falsificado dispare una retransferencia no autorizada
MNAME del SOA un NS público	Normal en hidden-signer — el MNAME identifica el origen de los cambios, no implica reachability pública
Backup del <code>key-directory</code> completo	Incluye el estado de KASP, no solo las claves — necesario para no perder el historial de rollover

- El signer nunca aparece en el NS RRset ni en el glue del padre — solo `ns2` y `ns3` lo hacen.
- La migración de un primario público a hidden signer es, en esencia, la misma disciplina de coherencia de NS/glue del módulo 3, aplicada a un cambio real: agregar el nuevo público antes de sacar el viejo.
- Siguiente: [07-DNSSEC-Avanzado.md](#) — NSEC3 y la mecánica completa de rollovers gestionados por KASP.

9 07 - DNSSEC avanzado

En el [módulo 4](#) dejamos dos cosas pendientes a propósito: la mecánica interna de un rollover, y NSEC3. Este módulo cierra ambas, y de paso agrega tres temas que hasta ahora no aparecieron: algorithm rollover, CDS/CDNSKEY, y multi-signer. Seguimos operando sobre `example.com`, con la misma `dnssec-policy "operadores"` del módulo 4, corriendo en el hidden signer (192.0.2.1) del [módulo 6](#).

9.1 NSEC3 en profundidad

En el [módulo 2](#) quedó dicho sin profundizar: NSEC3 existe para evitar *zone walking* — que alguien recorra los NSEC de una zona, uno detrás del otro, y reconstruya la lista completa de nombres que existen. NSEC3 resuelve el mismo problema que NSEC (probar que un nombre *no* existe), pero en vez de encadenar nombres en texto plano, encadena sus **hashes**.

9.1.1 nsec3param: iterations, salt, opt-out

Dentro de una `dnssec-policy`, NSEC3 se activa declarando `nsec3param`:

```
nsec3param iterations 0 optout no salt-length 0;
```

- **iterations** — cuántas veces se aplica el hash antes de guardarlo. Parece intuitivo pensar “más iteraciones, más seguro”, pero [RFC 9276](#) (BCP 236) cambió esa recomendación: iteraciones extra aumentan el costo de CPU para firmar *y* para validar, con beneficio de seguridad marginal — y ese costo extra es aprovechable para un ataque de agotamiento de CPU contra validadores. La recomendación actual es `iterations 0`. BIND, además, rechaza directamente valores mayores a 150.
- **salt-length** — un valor al azar mezclado en el hash para invalidar tablas precalculadas (rainbow tables). El mismo RFC 9276 recomienda **no usar salt** (`salt-length 0`) — la práctica mostró que no aporta protección real contra un atacante que ya puede consultar la zona directamente, y sí agrega complejidad operativa (cada rotación de salt cambia todos los hashes de la zona).

- **optout** — pensado para zonas con **muchas delegaciones sin firmar**: un registrador o un operador de subdominios que delega miles de subzonas, la mayoría todavía sin DNSSEC. Sin opt-out, cada una de esas delegaciones necesita su propio NSEC3 igual que un nombre firmado. Con **optout yes**, las delegaciones inseguras quedan afuera de la cadena NSEC3, y el tamaño/costo de firma de la zona no crece con ellas. **example.com** no tiene delegaciones propias, así que **optout** no es lo correcto acá — la opción existe para cuando sí las hay.

En la política **operadores** del módulo 4, la ausencia de **nsec3param** implicaba NSEC. Agregarlo es lo único que cambia:

```
dnssec-policy "operadores" {
    keys {
        ksk lifetime unlimited algorithm ecdsap256sha256;
        zsk lifetime P90D algorithm ecdsap256sha256;
    };

    dnskey-ttl PT1H;
    publish-safety PT1H;
    retire-safety PT1H;
    signatures-refresh P5D;
    signatures-validity P2W;
    signatures-validity-dnskey P2W;
    max-zone-ttl P1D;
    purge-keys P90D;

    nsec3param iterations 0 optout no salt-length 0;
};
```

9.1.2 Verificación

```
$ named-checkconf
$ sudo rndc reload example.com
zone example.com/IN: reloaded

$ dig @192.0.2.1 example.com NSEC3PARAM +dnssec +short
1 0 0 -
```

1 0 0 - es: algoritmo de hash 1 (SHA-1, el único que define el estándar NSEC3), 0 flags, 0 iteraciones, y - como salt (vacío) — exactamente lo que pide RFC 9276.

Una consulta por un nombre que no existe ahora trae NSEC3 en vez de NSEC, con el *owner name* hashado:

```
$ dig @192.0.2.1 nope.example.com A +dnssec
```

```
;; AUTHORITY SECTION:
```

```
b4c4z1n8h3k2m9p0q7r5s3t1u8v6w4x2.example.com. 3600 IN NSEC3 1 0 0 - ( c9d7e5f3g1h8i6j4k2l0m8n
```

```
b4c4z1n8h3k2m9p0q7r5s3t1u8v6w4x2.example.com. 3600 IN RRSIG NSEC3 13 3 3600 ( ... )
```

Compará ese `b4c4z1n8...` con el registro `example.com. NSEC www.example.com. ...` que vimos en el módulo 2: mismo propósito (probar la no-existencia de `nope.example.com` y delimitar qué nombres sí hay), pero acá nadie puede leer directamente qué nombre real corresponde a ese hash sin fuerza bruta.

9.2 Mecánica completa de rollovers

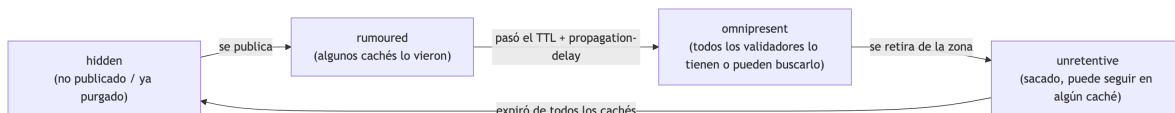
El módulo 4 dijo “KASP rota la ZSK cada 90 días sin intervención” y dejó ahí. Ahora sí: cómo es ese proceso por dentro.

9.2.1 Dos capas, no una

Hay que distinguir dos cosas que a menudo se mezclan bajo la palabra “rollover”:

1. **La línea de tiempo de una clave** — cuándo se generó, cuándo se publicó, cuándo empezó a firmar, cuándo se retiró, cuándo se borró del disco. Esto es lo que ves con `dnssec-settime`.
2. **La máquina de estados de cada *registro* que esa clave sostiene** — el DNSKEY, el DS (en el padre), y las RRSIG que esa clave produce. Cada uno de estos avanza, de forma independiente, por cuatro estados: **hidden** (no está en la zona ni en ningún caché) → **rumoured** (ya está publicado, pero no todos los cachés lo vieron todavía) → **omnipresent** (todo validador que pueda necesitarlo, ya lo tiene o puede obtenerlo) → **unretentive** (se sacó de la zona, pero puede quedar en algún caché todavía) → vuelve a **hidden**.

Es la capa 2 la que realmente determina *cuándo* KASP se anima a dar el siguiente paso — no alcanza con que pasen los días de la política, cada transición espera a que el estado anterior llegue a **omnipresent** (o **hidden**, según el caso) antes de seguir. Esto es lo que garantiza que nunca haya una ventana donde un validador confíe en algo que ya no está, o rechace algo que todavía no vio.



9.2.2 ZSK: estrategia pre-publish

La ZSK (41230) sigue la que ISC llama *pre-publish*: no hay doble firma de las RRSIG de la zona en ningún momento — lo que hay es un período de **doble DNSKEY**:

1. Se genera y publica el DNSKEY de la ZSK sucesora, todavía sin firmar nada con ella. Estado del DNSKEY: **hidden** → **rumoured**.
2. Se espera a que ese DNSKEY llegue a **omnipresent** (TTL del DNSKEY + margen de **publish-safety**) — recién ahí cualquier validador que necesite esa clave puede encontrarla.
3. **named** empieza a firmar la zona con la ZSK nueva; la vieja deja de firmar. Las RRSIG viejas quedan en **unretentive** hasta vencer de los cachés.
4. Una vez que las RRSIG viejas y el DNSKEY viejo llegan a **hidden**, se borran del disco (**purge-keys**, 90 días en nuestra política).

Como todo esto pasa dentro de la zona misma, sin depender del padre, KASP lo hace de punta a punta sin intervención — la ZSK 41230 del módulo 4, generada el 5 de agosto, con **lifetime P90D**, va a arrancar su primer rollover automático alrededor del 3 de noviembre.

9.2.3 KSK: estrategia double-KSK

La KSK (54915) es distinta, y acá sí hay **doble firma real**: durante la transición, el DNSKEY RRset queda firmado simultáneamente por la KSK vieja y la nueva.

1. Se genera y publica el DNSKEY de la KSK sucesora, y **enseguida** empieza a firmar el DNSKEY RRset junto con la KSK vieja — las dos RRSIG sobre DNSKEY conviven (esto es lo que vimos en el módulo 4 al migrar: “si ves dos KSK del mismo algoritmo, hay un rollover en curso”).
2. Cuando el DNSKEY y la RRSIG nuevos llegan a **omnipresent**, se pueden publicar CDS/CDNSKEY (siguiente sección) y el operador sube el DS nuevo al padre — **doble DS**, viejo y nuevo conviviendo en el padre durante la transición.
3. Recién cuando el operador confirma (**rndc dnssec -checkds**, ver más abajo) que el DS viejo se retiró del padre, la KSK vieja pasa a **unretentive** y eventualmente **hidden**.

Este último paso — subir/retirar el DS — es la única parte de todo el ciclo que KASP no puede hacer solo, porque no tiene acceso al panel del registrador. Por eso, como ya vimos en el módulo 4, la KSK de **operadores** tiene **lifetime unlimited**: no hay rollover de KSK programado, se dispara a mano cuando haga falta.

9.3 Inspeccionar y operar rollovers en vivo

9.3.1 dnssec-settime: la línea de tiempo

```
$ dnssec-settime -p all /etc/bind/keys/example.com/Kexample.com.+013+41230.key
Created:                Wed Aug  5 09:12:00 2026
Publish:                Wed Aug  5 09:12:00 2026
Activate:               Wed Aug  5 10:12:00 2026
Revoke:                 UNSET
Inactive:               Mon Nov  2 09:12:00 2026
Delete:                 UNSET
```

Esto es la capa 1 — cuándo pasa cada cosa. Para ver la capa 2 (el estado de cada registro), hay que mirar el archivo `.state` que KASP mantiene junto a la clave:

```
$ cat /etc/bind/keys/example.com/Kexample.com.+013+41230.state
This is the state of key 41230, for example.com.
Algorithm: 13
Length: 256
Lifetime: 7776000
KSK: no
ZSK: yes
Generated: 20260805091200
Published: 20260805091200
Active: 20260805101200
DNSKEYChange: 20260805101200
ZRRSIGChange: 20260805111200
DNSKEYState: omnipresent
ZRRSIGState: omnipresent
GoalState: omnipresent
```

DNSKEYState/ZRRSIGState: omnipresent con GoalState: omnipresent es justamente lo que se espera de una ZSK activa y estable — no hay rollover en curso, todo lo que debería estar publicado y confiable, lo está.

9.3.2 rndc dnssec -status: la vista operativa

En vez de ir archivo por archivo, `rndc dnssec -status` da el resumen — es la herramienta del día a día:


```
$ sudo rndc dnssec -status example.com
dnssec-policy: operadores
current time:  Mon Oct 26 14:00:00 2026

key: 41230 (ECDSAP256SHA256), ZSK
  published:      yes - since Wed Aug  5 09:12:00 2026
  zone signing:   yes - since Wed Aug  5 10:12:00 2026
```

```
Key will retire on Mon Nov  2 09:12:00 2026
- goal:          hidden
- dnskey:        omnipresent
- zone rrsig:    omnipresent
```

```
key: 54915 (ECDSAP256SHA256), KSK
  published:      yes - since Wed Aug  5 09:12:00 2026
  key signing:    yes - since Wed Aug  5 09:12:00 2026
```

```
No rollover scheduled
- goal:          omnipresent
- dnskey:        omnipresent
- ds:           omnipresent
- key rrsig:     omnipresent
```

A esta altura (fines de octubre), la 41230 ya avisa que se retira el 2 de noviembre — su sucesora todavía no aparece en el listado porque KASP recién la va a generar cuando el calendario de `publish-safety` lo indique, no con meses de anticipación.

9.3.3 Forzar y confirmar un rollover a mano

Para no esperar al calendario — por ejemplo, ante sospecha de compromiso de una clave — se fuerza con `-rollover`:

```
$ sudo rndc dnssec -rollover -key 41230 example.com
```

Y para la parte del rollover de KSK que KASP no puede resolver solo — confirmar que el DS ya cambió en el padre —, `-checkds`:

```
$ sudo rndc dnssec -checkds -key 12345 published example.com    # el DS nuevo ya está en el p
$ sudo rndc dnssec -checkds -key 54915 withdrawn example.com    # el DS viejo ya se retiró d
```

Cada uno de estos comandos es, en esencia, la versión operable de un paso que en el módulo 4 hicimos “a ojo” (esperar y volver a correr `delv`) — acá se lo confirmás a KASP explícitamente para que el rollover avance al siguiente estado.

9.4 Algorithm rollover

Todo lo anterior asume que el algoritmo (ECDSAP256SHA256) no cambia — solo cambian las claves. Un algorithm rollover es distinto: cambia el algoritmo mismo, algo que se hace para dejar atrás un algoritmo débil o deprecado (el caso típico: una instalación vieja que todavía firma con RSASHA256 y hay que migrarla).

Supongamos que, antes de este curso, `example.com` firmaba con este perfil:

```
dnssec-policy "operadores" {
    keys {
        ksk lifetime unlimited algorithm rsasha256 2048;
        zsk lifetime P90D algorithm rsasha256 1024;
    };
    ...
};
```

Migrar a `ecdsap256sha256` (la política real que venimos usando desde el módulo 4) es, en la configuración, un solo cambio de línea. Lo que dispara ese cambio es más que un simple rollover de clave:

- KASP genera un KSK y una ZSK nuevas, en el algoritmo nuevo — no reemplaza las viejas, **conviven** con ellas.
- Durante la transición, la zona queda firmada con **los dos algoritmos en simultáneo**: cada RRset lleva una RRSIG con algoritmo 8 (RSASHA256) y otra con algoritmo 13 (ECDSAP256SHA256). `dnssec-verify` lo muestra así:

```
$ dnssec-verify -o example.com /var/cache/bind/example.com.signed
Verifying the zone using the following algorithms: RSASHA256, ECDSAP256SHA256.
Zone fully signed:
Algorithm: RSASHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                  ZSKs: 1 active, 0 stand-by, 0 revoked
Algorithm: ECDSAP256SHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                  ZSKs: 1 active, 0 stand-by, 0 revoked
```

- El DS en el padre también pasa por su propio período doble (mismo mecanismo double-KSK de la sección anterior, mismo `rndc dnssec -checkds` para confirmarlo).

- Recién cuando las claves y firmas del algoritmo viejo llegan a **hidden**, KASP las retira — la zona queda firmada únicamente con ECDSAP256SHA256.

La razón de este doble período es la misma que motiva toda la máquina de estados: mientras haya un validador ahí afuera que todavía no vio el algoritmo nuevo (o que dejó de confiar en el viejo antes de tiempo), sacar cualquiera de los dos de golpe rompe la validación para esa porción de internet.

9.5 CDS/CDNSKEY

Publicar el DS en el padre, como lo hicimos en el módulo 4, es un paso manual: generarlo con `dnssec-dsfromkey` y copiarlo al panel del registrador. CDS y CDNSKEY existen para automatizar ese paso — son registros que la zona hija publica en **sí misma**, con el contenido que el DS *debería* tener, para que el padre (o el registrador, actuando en su nombre) los consulte y actualice el DS solo.

Se activa en la política:

```
dnssec-policy "operadores" {
    ...
    cdnskey yes;
    cds-digest-types { "SHA-256"; };
};
```

Con esto, el archivo `.state` de la KSK suma un campo `PublishCDS`, que marca cuándo es seguro publicarlo — el mismo criterio de **omnipresent** que ya vimos, aplicado a esta decisión: no tiene sentido publicar un CDS que apunta a una KSK que todavía no llegó a todos los cachés.

```
$ dig @192.0.2.1 example.com CDS +short
54915 13 2 A94C3B1F...
$ dig @192.0.2.1 example.com CDNSKEY +short
257 3 13 ...base64...
```

Esto no reemplaza el flujo de `-checkds` de la sección anterior — lo complementa. `rndc dnssec -checkds` es cómo el operador (o un **parental-agents** automatizado) le confirma a KASP que el DS ya cambió en el padre; CDS/CDNSKEY es cómo la zona le informa al padre qué DS *debería* publicar. La limitación real es de soporte: no todos los registradores ni todos los TLD hacen polling de CDS/CDNSKEY — antes de depender de esto en producción, hay que confirmar que el registrador de `example.com` lo soporta.

9.6 Multi-signer (RFC 8901, Model 2)

Todo lo visto hasta acá — incluida la arquitectura hidden-signer del módulo 6 — sigue siendo **un solo proveedor DNS**, con un único signer (redundado con `ns2/ns3`, pero un único origen de firma). Multi-signer va un paso más allá: dos o más proveedores DNS **independientes**, cada uno con su propio hidden signer y sus propias claves, firmando y sirviendo la misma zona — pensado para tolerar que un proveedor entero (no solo un servidor) tenga una caída o un incidente.

RFC 8901 define dos modelos; el que soporta BIND vía `dnssec-policy` es el **Model 2**: cada proveedor genera y gestiona su propio KSK/ZSK, sin compartir material privado entre proveedores (Model 1, donde los proveedores comparten claves, existe en el papel pero casi nadie lo usa — compartir una clave privada entre organizaciones distintas es exactamente lo que la arquitectura del módulo 6 busca evitar). Lo que sí se comparte es información pública: cada proveedor tiene que conocer e incluir en su propio DNSKEY RRset las claves públicas de los demás proveedores (importadas por actualización dinámica), y el DS publicado en el padre tiene que listar las KSK activas de **todos** los proveedores a la vez.

La parte operativamente difícil no es la firma en sí — es la coordinación: un rollover de cualquiera de los proveedores obliga a sincronizar el DNSKEY RRset combinado entre todos ellos antes de que ese rollover pueda avanzar, y el DS en el padre tiene que reflejar exactamente el conjunto vigente en cada momento. Es una arquitectura que tiene sentido para operadores grandes que ya dependen de más de un proveedor DNS por razones de resiliencia — no es algo para activar por default en un `example.com` de un solo proveedor.

9.7 Resumen

Concepto	Qué logra
<code>nsec3param iterations 0 optout no salt-length 0</code>	NSEC3 con los parámetros que recomienda RFC 9276 — evita zone walking sin costo extra de CPU
Estados por registro (hidden/rumoured/omnipresent/unretentive)	Lo que realmente determina cuándo KASP avanza un rollover — no el simple paso del tiempo
Pre-publish (ZSK) vs. double-KSK (KSK)	La ZSK duplica el DNSKEY antes de firmar; la KSK firma con ambas claves a la vez y duplica el DS en el padre
<code>rndc dnssec -status / -rollover / -checkds</code>	Ver el estado de un rollover, forzarlo fuera de calendario, y confirmarle a KASP que el DS ya cambió en el padre

Concepto	Qué logra
Algorithm rollover	Cambiar el algoritmo en la política dispara una transición con firma dual (viejo + nuevo algoritmo), gestionada con la misma máquina de estados
<code>cdnskey yes + cds-digest-types</code>	La zona publica lo que el DS debería ser, para que el padre lo sincronice solo (si el registrador lo soporta)
Multi-signer (RFC 8901 Model 2)	Redundancia entre proveedores DNS independientes — BIND lo soporta, pero la coordinación de DNSKEY/DS es responsabilidad del operador

Con esto se cierra el temario de 7 módulos: desde por qué existe DNSSEC ([módulo 1](#)) hasta poder operar, diagnosticar y hacer evolucionar una zona firmada en producción, con la arquitectura y el nivel de detalle que un operador — no solo alguien que la configuró una vez — necesita.